

My SubTitle



date and time

Contents

A.1 Start	1
[1] Run examples	1
[2] Typical IDE layout	1
[3] Portable-rivt	2
[4] Codespace-rivt	3
[4] System-rivt	4
rivt-uv	4
A.2 Motivations	6
[1] Background	6
[2] Use Cases	7
A.3 FAQ	7
[1] Contact	7
[2] Licenses	7
[3] <i>rivt</i> design and development	8
B.1 Overview	9
[1] Summary	9
[2] rivt docs	9
[2] rivt API	10
[3] Files/Folders	10
[5] Framework	11
B.2 Framework	12
[1] Install <i>rivt-framework</i>	12
C.1 Docs	12
[1] Doc Files	12
[2] Reports	13
C.3 Files	13
[1] File Types	13
C.2 Folders	16
[1] rivt Folder	16
[2] Folder Names	18
C.4 Reports	19
[1] Report compilation	19
[2] rivt-report.py	19
[3] conf.py	21
C.5 Example	24

[1] rvt file	24
D.1 Markup	26
[1] API functions	26
[2] rvt string	26
[3] Tags and Commands	28
[4] Folders	29
D.2 Line Tags	30
[1] Line Tag Summary	30
[2] Center text	30
[3] Right justify text	31
[4] Text math	31
[5] LaTeX math	31
[6] Endnote number	32
[7] Glossary link	32
[8] Section link	32
[9] Doc link	33
[10] URL link	33
[11] Variable value	34
[12] Equation number	34
[13] Table number	35
[14] Figure number	35
D.3 Block Tags	35
[1] Block Tag Summary	35
[2] Shell script	36
[3] Indent text	36
[4] Indent italic	37
[5] Endnotes	37
[6] Text	38
[6] Text	38
[7] Topic	39
[8] Box	40
[9] Python	40
[10] Markup block	41
[11] Metadata	41
[12] Layout block	42
[13] End block	42
D.4 Commands	43

[1] Command Summary	43
[2] Shell file	44
[3] Text file	44
[4] Table file	45
[5] Image file	45
[6] Adjacent images	46
[7] Valtable file	46
[8] Define value	47
[9] Assign value	47
[9] Function value	48
[10] Compare values	48
[11] Python file	49
[12] Markup file	49
[13] Attach PDF	50
[14] Publish doc	50
D.5 Run - rv.R	50
[1i] rv.R Markup	50
[2i] Block Tags	51
[3i] Commands	51
D.6 Insert - rv.I	51
[1i] rv.I Markup	51
[2i] Line Tags	51
[3i] Block Tags	51
[4i] Commands	51
D.7 Values - rv.V	52
[1i] rv.V Markup	52
[2i] Line Tags	52
[3i] Block Tags	52
[2i] Commands	52
D.8 Tool - rv.T	53
[1i] rv.T Markup	53
[2i] Block Tags	53
[3i] Commands	53
D.9 Doc - rv.D	54
[1i] rv.D Markup	54
[2i] Block Tags	54
[3i] Commands	54

E.1 Collaboration	54
[1] Overview	54
[2] Open Models	55
[3] Git and GitHub	55
[4] VSCode	56
E.2 Public files	56
[1] Overview	56
[2] Public Folder	57
[3] Example	57
E.3 Git	57
[1] Git	57
E.4 GitHub Search	57
[1] GitHub Global Search	57
[2] GitHub Organization Search	58
F.1 Quick Ref	58
[1] API	58
[2] Line Tags	59
[3] Block Tags	59
[4] Commands	60
[5] Folders	62
[6] Python Requirements	63
[7] VSCode Settings	65
F.2 Terms	69
[1] rivt Terms	69
[2] Python Terms	71
[3] GitHub and VSCode Terms	71
Index	73



rivt is an extensible, open source Python tool for writing and organizing engineering documents. A *rivt* document, referred to as a *doc*, is published as a text, HTML or PDF file from a *rivt* file. A *rivt file* is a Python file (.py) that imports the [rivtlib](#) Python package and includes *rivt* markup.

Multiple *rivt docs* in a *rivt folder* may be compiled and linked into a *rivt* report. This site is an example of an HTML *rivt report*.

An interface for finding public *rivt files* on *GitHub* is [here](#) and a collection of open source models and calculations on Google Drive, including *rivt*, is [here](#).

A.1 Start

[1] Run examples

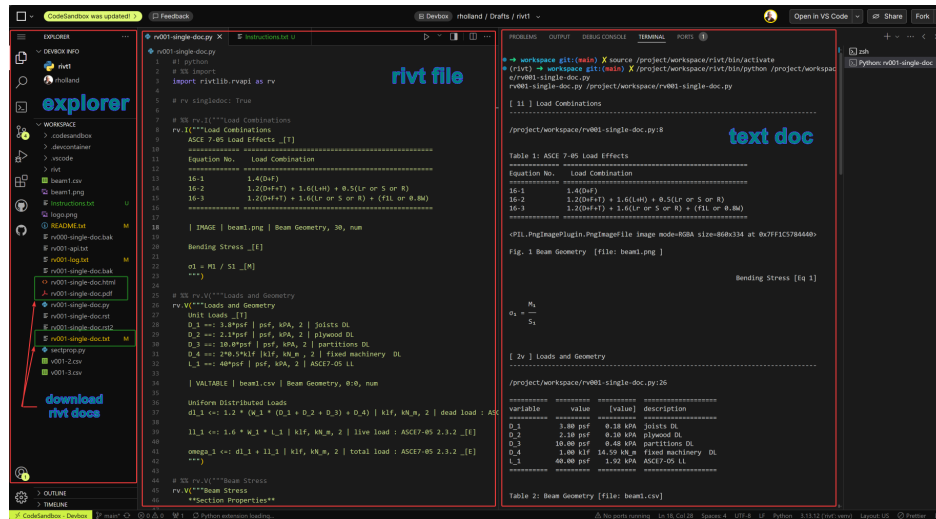
This section outlines steps for installing *rivt* and running examples. *rivt* is an open source project and, like all open source projects, is built from a variety of components written by many contributors. Most of the components are Python libraries [VSCode](#) but effective use of *rivt* also requires separate programs like the IDE [VSCode](#). For a more general overview of *rivt* start [here](#).

There are three methods for installing and running *rivt* examples.

1. If you are not familiar with Python or programming *Portable-rivt* is the recommended method to download. This is a single zip folder that contains a full *rivt* and VSCode installation with examples. It is a single download for Windows and requires no installation. Further details are described [here](#)
2. If you are familiar with [GitHub](#) and have an account you can run a cloud version of *rivt* in your browser using [Codespace-rivt](#). Further details are described [here](#).
3. Finally, if you are familiar with Python and want the most flexible and extensible environment, the recommended method is to install *rivt* at the system level in an isolated environment. *System-rivt* is described [here](#).

[2] Typical IDE layout

[3] Portable-rivt



rivt can be run from the command line but efficient editing and running of *rivt* files is typically done in an IDE. A typical **IDE** layout can be viewed at

[Codesandbox](#).

This explorable browser example shows the *rivt* file in the left panel, the text *doc* output in the right panel or second tab, and the file explorer in the far left panel. Docs (txt, PDF and HTML output files) can be downloaded from the file explorer including the text, pdf and html reports. The IDE cannot be used to run files. Setting up a GitHub account and forking Codespace-rivt is needed.

[3] Portable-rivt

A *rivt-portable* installation is recommended for Windows users unfamiliar with Python. *rivt-portable* is a zip file :

```
win64-rivt-portable-n.n.n[an].zip
```

where n is a number representing the major, minor and patch release number. An *an* appended to the version is an *alpha* release where users should expect that features are missing and *rivt markup* syntax may change in the future. Zip file contents must be unzipped into a directory with read-write access - typically the users home folder or a flash drive.

The zipped and unzipped file sizes include the following contents and are approximately 1GB and 2GB respectively:

```
release: rivt-portable-1.0.0a4.zip
```

1. Python 3.14 with rivt packages
2. VSCode 1.109 with rivt extensions
3. rivt-1.1.0a4
4. Example rivt files

Releases may be download from the [GitHub repository](#).

[4] Codespace-rivt

The primary advantages of *rivt-portable* are

1. simplified installation
2. package integration
3. isolation from system files.

After unzipping, VSCode and examples are started with a single shortcut.

The primary disadvantages of *rivt-portable* are that *rivt* packages and extensions must be updated by updating the zip file as a whole, and it is more difficult to integrate with other Python programs.

[4] Codespace-rivt

VSCode can be run locally or in the cloud. For local installation see [5] Framework.

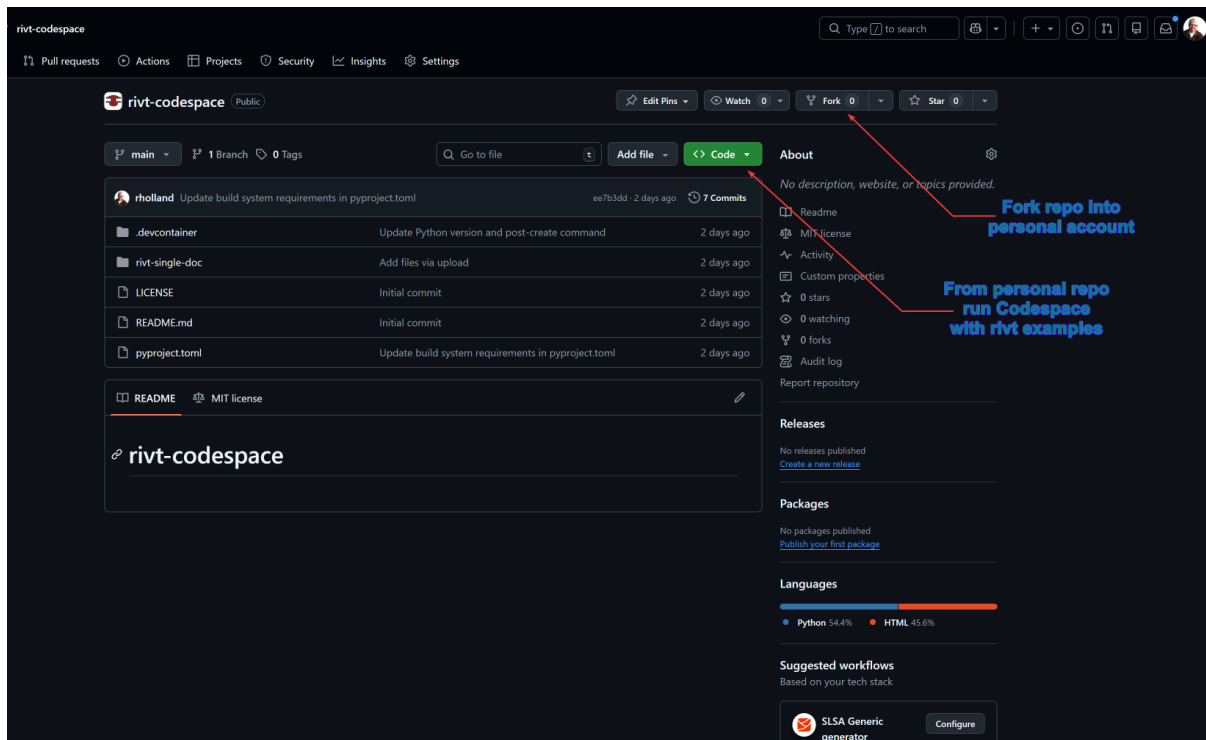
The cloud version is a [Codespace](#). A rivt Codespace is a VSCode cloud environment with *rivt* extensions for editing and running *rivt files*. It can be forked (copied) into a personal Github account. Github setup is described [here](#).

A rivt Codespace can be forked from [this repository](#) by following the steps below. Example rivt files are included.

Fork a rivt Codespace

1. Click the Fork button in the upper right corner.
This creates a copy of the repository in your GitHub account.
2. Switch to your GitHub account and open the forked repository
3. Click the green Code button
4. Select the Codespaces tab
5. Click the rivt-codespace-examples option.
6. This creates a Codespace environment in your GitHub account and opens the repository in the online VSCode IDE. The environment includes the rivt packages and extensions needed to run and publish rivt docs and files.

[4] System-rivt



Local installation methods

1. Windows portable zip folder ([3] Portable-rivt) *rivt-portable* is recommended for Windows users unfamiliar with Python.
2. System Python ([4] System-rivt) with or without uv manager (rivt-uv)

[4] System-rivt

rivt may be installed at the system level.

1. Install Python using [Python installers](#) The minimum version required by *rivt* is Python 3.14.
2. Install *rivtlib* and dependencies using pip

```
pip install rivtlib
```

A list of the dependencies is here.

rivt-uv

Rather than installing *rivtlib* at the system level, it can be installed in an isolated environment using the [uv package manager](#). The primary advantage of *rivt-uv* is the simplicity of updating packages while keeping them isolated from the system Python.

[4] System-rivt

rivt-uv installation is recommended for users with some familiarity with Python and programming.

Step 1. Install uv

Different methods for installing *uv* are described [here](#).

The recommended method for installing *uv* is from the command line:

Windows:

```
winget install --id=astral-sh.uv -e  
  
or  
  
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex
```

macOS and Linux:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Step 2. Create the rivt environment

After installing *uv*, download and run the following rivt install script. It installs an isolated *rivt environment* and example files in a folder named *rivt-examples* in the users *Home* directory.

Windows: rivtuv.cmd

OSX and Linux: rivtuv.sh

Step 3. Run the example program from Pyzo.

The *rivt environment* is activated by:

Windows:

```
./venv/Scripts/activate
```

macOS and Linux:

```
source venv/bin/activate
```

rivt includes the [Pyzo](#) IDE for editing and running examples. Typing the command *pyzo-example* from the environment root will load the example file *rv00-simple-doc.py* where it can be edited and run.

The example file can also be run from the command line with the command:

```
python -m rivt rv001-single-doc.py
```

The *uv* environment can be completely removed with the following commands.

```
uv deactivate
rmdir /s /q rivt-examples
uv cache clean
```

A.2 Motivations

[1] Background

rivt is an open source software project that simplifies sharing and reuse of engineering documents. This has always been a challenge because the inclusion of text, images, tables, calculations, models and computer code required in engineering reports has led to development of incompatible, mutually exclusive software. Sharing and reuse has been restricted by design and terms of use of existing software. This includes:

- incompatible documents from different programs
- frequent software updates needed to maintain document access
- high update costs
- software limited to specific platforms
- limited document version control
- limited report generation features
- limited collaboration features

rivt is designed to address these limitations and function as a complement or replacement to existing software. The table below summarizes and compares limitations between different software programs.

Software Comparison

Program	Rep ¹	Ver ²	Txt ³	Comp ⁴	CP ⁵	Collab ⁶
Matlab	no	no	no	no	no	no
Mathcad	no	no	no	no	no	no
Mathematica	no	no	no	no	no	no
Cloud SaaS	limited	no	no	no	yes	limited
Excel	limited	no	no	yes	no	yes
Jupyter	no	no	no	yes	yes	yes
rivt	yes	yes	yes	yes	yes	yes

- 1 Report generation
- 2 Native version control
- 3 Plain text, readable input files
- 4 Forward and backward compatibility

- 5 Cross-platform
- 6 Collaboration support

[2] Use Cases

The primary use case for *rivt* is producing engineering documents that lie between back of envelope calculations and sketches, and formal journal publications. In other words, it produces clear, formatted, organized documents that are easy to write and edit.

The second use case is when flexibility is needed to produce documents in a variety of formats including text, PDF or HTML.

A third use case is when documents need to interact with other software programs.

rivt files can function as a front and back end for:

1. software control
2. visualization
3. instrumentation

rivt docs can be used for:

1. internal communication
2. research documentation
3. government permits
4. technical reports
5. funding applications

Because *rivt* is compatible with collaborative tools it may be used for:

1. teaching
2. presentations
3. real time collaboration

A.3 FAQ

[1] Contact

support@rivt.info

[2] Licenses

What is the open source license for *rivtlib*?

rivtlib is distributed under the [MIT open source license](#).

Which open source licenses applies to *rivt*?

The Python packages that are part of the *rivt project* are distributed primarily under the BSD, Apache and MIT open source licenses. Python itself is distributed under the Python Software Foundation License (PSFL).

Which open source licenses apply to *rivt* files?

The open source license for a public *rivt* file is specified in the `__[[Metadata]]` block tag. The default license is the MIT license.

Which open source licenses apply to the *rivt* framework?

VSCode - Visual Studio Code uses a proprietary Microsoft license for the compiled product, allowing free use for development, testing and commercial applications. The source code ("Code - OSS") is under the MIT license. You can use VS Code freely for any purpose, including commercial projects, but some bundled extensions or specialized components (connections to the Microsoft servers and marketplace) may have different terms. Most extensions in the Marketplace use the MIT license.

VSCodium is a fully open source alternative that removes Microsoft's proprietary components and built-in telemetry. However its extension marketplace is less complete.

LaTeX - LaTeX is released under the LaTeX Project Public License (LPPL) for LaTeX components and MIT and BSD variants for many core utilities and packages.

QCAD - The QCAD source code is distributed under the open source license GPLv3 with exceptions to allow proprietary extensions.

Git - The Git software is released under the GNU General Public License Version 2.0 (GPLv2).

[3] *rivt* design and development

What is the *rivt* release schedule?

1.0.x bug releases are issued as needed.

1.x.0 minor releases are planned monthly following the release of version 1.0.0

X.0.0. major releases will have backward compatibility. See [past releases](#) and [changelog](#) .

What are the programs and libraries used to build *rivt*?

The *rivt project* uses dozens of Python libraries (see [6] Python Requirements):

- numpy, sympy and related scientific computing libraries
- docutils and related text processing libraries
- Reportlab, rst2pdf and related PDF generation libraries
- Sphinx, Pydata and related HTML libraries

The *rivt framework* uses the following open source and free programs:

- [VSCode](#) for editing
 - [LaTeX](#) for PDF generation
 - [QCAD](#) for diagramming
 - [Git](#) for document sharing and version control
-

B.1 Overview

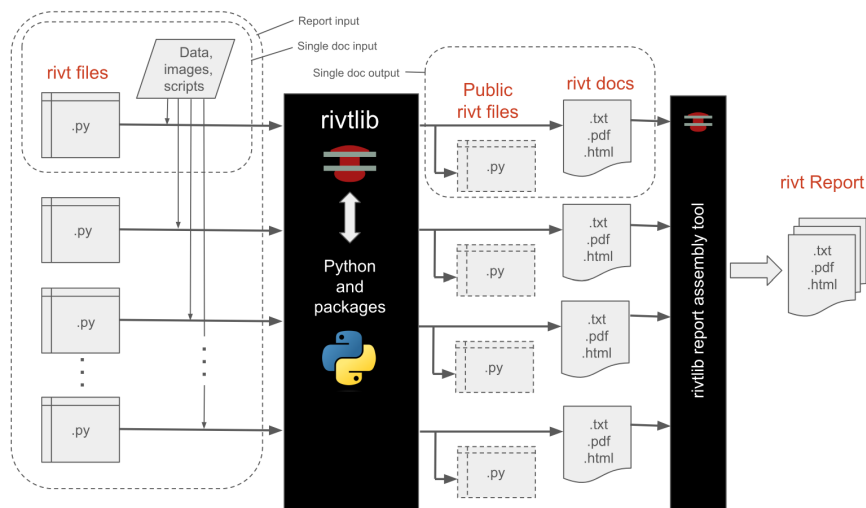
[1] Summary

rivt is an open source Python project that imports the [rivtlib Python package](#) and dependencies ([6] Python Requirements). A *rivt* file publishes a formatted *rivt* doc as a text, PDF or HTML file. A *rivt file* is a Python file (.py) that imports the *rivtlib* Python package and includes *rivt* markup. A collection of *rivt docs* may be assembled into a *rivt* report.

rivt files are generally edited and run in an IDE. The lightweight [Pyzo](#) IDE is installed with *rivtlib*. The [VSCode](#) IDE is a full featured IDE that is part of the *rivt* framework and included with [3] Portable-*rivt*.

rivt file examples are provided [here](#). An interface for searching *public rivt files* on *GitHub* is [here](#). A *public rivt file* is the parts of a *rivt file* the author chooses to share under an [Open Source license](#). *rivt* itself is distributed under the [MIT open source license](#). (see [2] Licenses).

[2] rivt docs



rivt Doc Processing

Each *rivt* file outputs a corresponding doc in the format specified in *rv.D()* API. A *rivt* file number has the form:

```
rvAnn-filename.py
```

where *rv* is a required prefix, *A* is an alphanumeric character and *nn* are two digit non-negative integers. The prefix is used to organize reports into divisions and subdivisions.

Corresponding *rivt* docs are output as:

```
rvAnn-filename.txt
rvAnn-filename.pdf
rvAnn-filename.html
```

A *rivt* report is organized using the *rivt file numbers*. If the *rivt* file names are:

[2] rivt API

```
rvA01-filename.py
rv105-filename.py
rv212-filename.py
```

the corresponding *doc numbers* in the would be:

- A.1 (division A, subdivision 1)
- 1.5 (division 1, subdivision 5)
- 2.12 (division 2, subdivision 12)

Docs are sorted alpha-numerically into divisions and subdivisions. A *doc number* corresponds to a report subdivision. Note that leading zeroes are dropped in the formatted output.

[2] rivt API

The *riv* API includes API functions, [3] Tags and Commands and files. The API is designed to be:

- lightweight

riv markup wraps reStructuredText and is made up of about 3 dozen tags and commands in total.

- **extensible**

rivlib is written in Python with direct access to thousands of Python packages. Python scripts and external programs can be run from *riv* files using the

- **flexible**

A *riv* file produces a text, HTML or PDF *doc*. Multiple *docs* can be organized into a report.

An API function starts in the first column and takes a triple quoted riv string (rS) containing riv markup as the argument. The first line of a *riv string* is a header substring, followed by a content substring indented 4 spaces for improved readability and section folding.

The *header substring* specifies the section title and other processing parameters. The content substring includes riv markup and other arbitrary text. For further details see here.

```
rv._("""Section Label | parameters
    Content text that is
    indented four spaces.
    ...
    """)
```

[3] Files/Folders

A riv file is a Python plain text file (*.py*) that includes *riv markup* and imports the rivlib package and API into the *rv* namespace:

```
import rivlib.rvapi as rv
```

[5] Framework

*rv*t files are stored in a root folder designated with the prefix *rv*t-. Each *rv*t file and corresponding *rv*t doc has a prefix that is used to organize the report. The prefix has the form

```
rvANN-filename.py
```

where A is an alpha-numeric character used for organizing report divisions, and NN is a two digit number used to organize sub-divisions. The file name is used as the *doc* title unless overridden in *rv.D*. Use hyphens rather than spaces to separate words in a file name. They are replaced with spaces when a *doc* title is extracted.

The top level folder structure is shown below. A more detailed description of the folder structure is here.

```
rvt-Report-Label/      Report Folder
├── .vscode/            optional VSCode settings
├── rvt-public_/        rvt-generated public files
│   ├── src/            source files
│   ├── README.txt      public report
│   ├── rv-101-filename1.py public rvt file
│   ├── rv-102-filename2.py public rvt file
│   ├── rv-201-filename3.py public rvt file
│   ├── rv-202-filename4.py public rvt file
│   └── ...
└── rvt-report/         rvt files and docs
    ├── _published/     published docs and reports
    ├── _rstdocs/       restructured text files
    ├── _stored/        stored files
    ├── src/            source files
    ├── rvt-report.py   report generating script
    ├── rv101-filename1.py rvt file
    ├── rv102-filename2.py rvt file
    ├── rv201-filename3.py rvt file
    ├── rv202-filename4.py rvt file
    └── ...
```

[5] Framework

The *rv*t framework includes separate programs that provide additional editing , formatting, version control, and diagramming tools.

Purpose	Link
Editing	VSCode
Editing Extensions	[7] VSCode Settings
Added Formatting	LaTeX
Version Control	Git
2D Diagramming	QCAD
3D Diagramming	FreeCad

B.2 Framework

[1] Install *rivt-framework*

The *rivt framework* includes separate programs that provide additional editing , formatting, version control, and diagramming tools.

Purpose	Link
Editing	VSCode
Editing Extensions	[7] VSCode Settings
Added Formatting	LaTeX
Version Control	Git
2D Diagramming	QCAD
3D Diagramming	FreeCad

The *rivt framework* includes installation of the following programs:

1. [VSCode](#) and [VSCode extensions](#). *rivt* extensions and snippets are here.
2. [Git](#) and set up a [GitHub](#)
3. [LaTeX](#)
4. [FreeCad](#)

C.1 Docs

[1] Doc Files

A *rivt doc* is the output of a *rivt file* processed by *rivtlib*. Each *rivt file* outputs a corresponding formatted doc with a *doc number* derived from the *rivt file* name. A *rivt report* is organized using the *rivt doc numbers*. The *rivt file* name has the form:

```
rvAnn-filename.py
```

where *rv* is a required prefix, *A* is an alphanumeric character and *nn* are non-negative integers.

```
import rivtlib.rvapi as rv
# rv public=True
```

If the *rivt file* names are:

```
rvA01-filename.py
rv105-filename.py
rv212-filename.py
```

[2] Reports

the corresponding *doc* references would be:

```
doc file names
rvA01-filename.type
rv105-filename.type
rv212-filename.type

doc references
A.1 (division A, subdivision 1)
1.5 (division 1, subdivision 5)
2.12 (division 2, subdivision 12)
```

Note that leading zeroes are dropped. *Docs* are sorted alpha-numerically into divisions and subdivisions in the *report*.

[2] Reports

Each *rivt* file outputs a corresponding *doc* written to the *publish* folder in the specified format. A *rivt report* is assembled from a set of *docs* using the *rivt-report.py* script. The *report folder structure* is described in [1] *rivt Folder*.

Docs may be output as text, HTML or PDF. PDF and HTML *docs* are produced using the *rst2pdf* and *Sphinx* Python libraries.

Reports are assembled from *docs* using the *rivt-report.py* script in the *Src/Tools* folder. A single *doc* that is not part of a report does not require running the *rivt-report.py* script.

C.3 Files

[1] File Types

rivt file

A *rivt file* provides the formatting instructions and content that generate *rivt docs*. Each *rivt file* has a *doc number* prefix that specifies the division and subdivision of the *doc*. *rivt reports* are assembled using *doc numbers*. The subdivision name is taken from the name that follows, unless over-ridden in the report script. *Division* names are specified in the *report script*.

The *doc number* prefix includes a single digit capital alphanumeric division label *D*, followed by a two digit subdivision number *ss*. For example:

```
[rvDss-]file-name.py

rvB02-Beam-Loads.py
```

where the default *doc* name is “Beam Loads” in subdivision 2 of division C. Hyphens are stripped from file name.

Note:

The following files are read and written by *rivtlib* when processing a *rivt file*. Files are labeled by origin and folder.

origin: author input

folder: *rivt-report-folder-name*

Refer to markup for *command* details.

Run file

A *run file* is a shell or command script executed using the | RUN | command.

origin: author input

folder: *src/* - or a user created subfolder of */src*

API function: *rv.R*

Insert file

An *insert file* is an image, table or text file executed using the | IMAGE | , | TABLE | , or | TEXT | command.

origin: author input

folder: *src/* - or a user created subfolder of */src*

API function: *rv.I*

Value file

A *value file* is a *csv* file that defines variable values. It is imported using the | VALUE | command. Values defined by equations and a `_[[VALUES]]` block are written to a file with the name composed of the *doc* and *section* number. The file may be read by other *rivt files*.

Read

origin: author input

folder: *src/Values* - or a user created subfolder of */src*

API function: *rv.V*

Write

origin: program output

folder: *src/Values*

API function: *rv.V*

Tool file

Tool files can be executed by | PYTHON | , | HTML | , | RST | or | LATEX | commands. They may be used for analysis and generation of text, tables, images and other formatted content. Executing a tool file may:

- define functions and classes for use in the *rivt file*
- generate images, tables and text for inclusion in the *doc*
- insert markup text into the *doc*

Read

origin: author input

folder: *src/Tools* or a user created subfolder of */src*

API function: *rv.T*

Write

origin: program output

folder: *src/Tools*

API function: *rv.T*

Doc file

Each *rivt* file outputs a *doc* file with the same *doc name and number* and the selected *doc type* suffix. *Doc* files are written to the *publish* folder as text, PDF or HTML files using | PUBLISH | command.

PDF files may be prepended or appended to the *doc* using the | APPEND | command.

origin: program output

folder: *src/* - or a user created subfolder of */src*

API function: *rv.D*

Report file

A *report* file is generated by the Python *report* script and is written to the designated *publish* subfolder. The *report* file name is taken from the *report* folder name unless specified in the *rivt-report.py* script.

origin: program output

folder: */publish/* subfolder

Report script

The *report* script is a Python file that assembles the *report*. It specifies the *docs* that should be included, whether the *rivt* files should be re-executed and other format settings.

origin: author input

folder: */publish*

Public rivt file

origin: program output

folder: */public*

A public rvt file is a copy of the *rvt file* that includes all of the sections marked as *public* in the *header*. It has the same name as the *rvt file* with an added hyphen between “rv” and the docnumber e.g. *rv-Dss-filename.py*

Log file

Log files are written to the *log folder*. They include:

- a backup of the rvt file
- a file listing the execution steps when processing the *rvt file*
- a file listing each API function call with the header.

origin: program output

folder: */logs*

C.2 Folders

[1] rvt Folder

Folder Names

Top Level Folders

[rvt-]Report-Label/	Report Folder
├── .vscode/	optional VSCode settings
├── README.txt	rvt-generated report
├── [_rvt-public]/	rvt-generated public files
│ ├── _src/	source files
│ ├── README.txt	public report
│ ├── rv-101-filename1.py	public rvt file
│ ├── rv-102-filename2.py	public rvt file
│ ├── rv-201-filename3.py	public rvt file
│ └── rv-202-filename4.py	public rvt file
│ ...	
└── [rvt-report]/	rvt files and docs
│ ├── [src_]/	source files
│ ├── [_published]/	published docs and reports
│ ├── [_rstdocs]/	restructured text files
│ ├── [_stored]/	stored files
│ ├── [config-report.py]	report generating script
│ ├── [rv101-]filename1.py	rvt file
│ ├── [rv102-]filename2.py	rvt file
│ ├── [rv201-]filename3.py	rvt file
│ └── [rv202-]filename4.py	rvt file
│ ...	

Expanded Folders

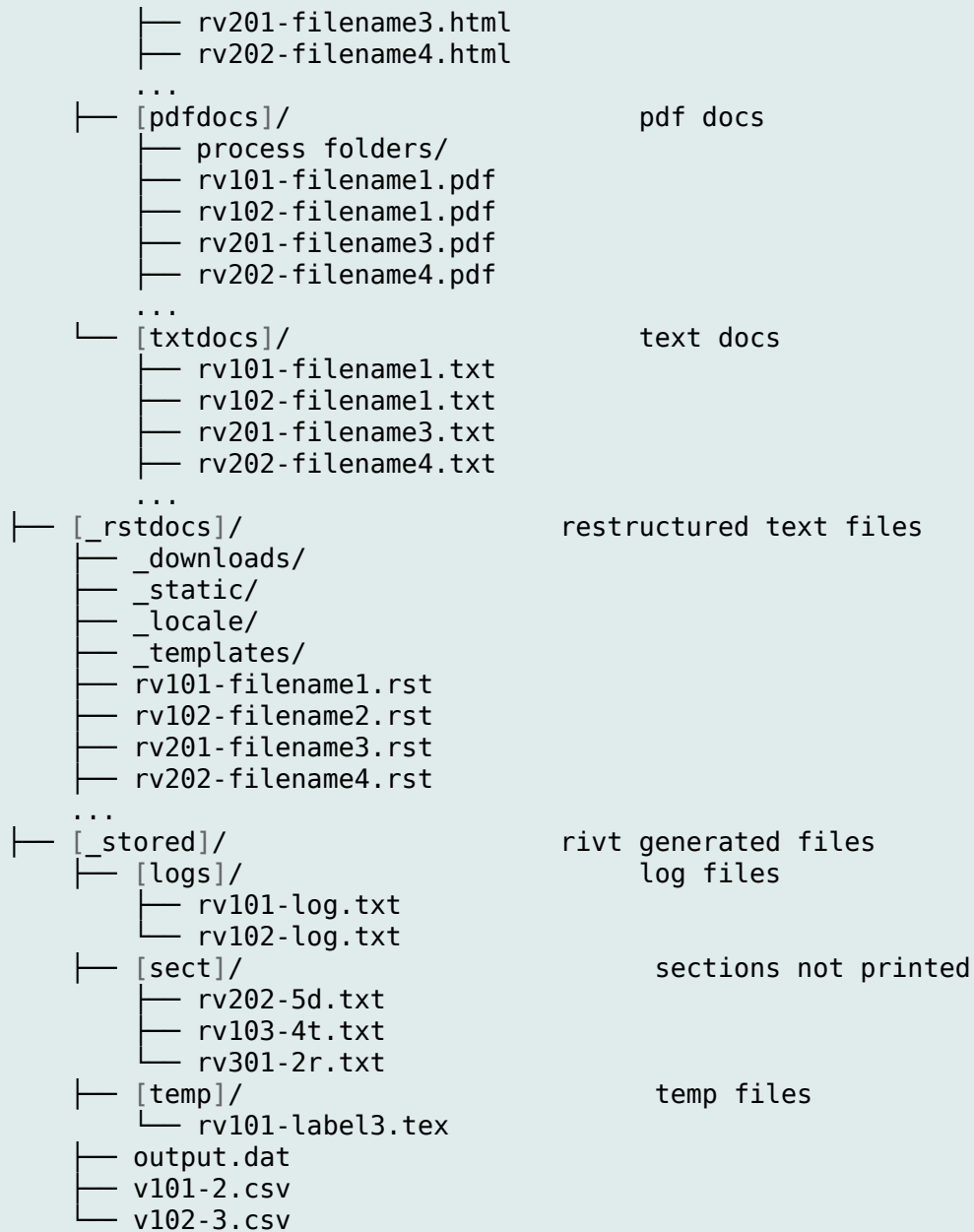
A typical rvt folder structure is shown below. The required *rvt file* names and prefixes are shown in brackets.

[rvt-]Report-Label/	Report Folder
├── .vscode/	optional VSCode settings

```

├── README.txt                rivt-generated report
├── [_rivt-public]/           rivt-generated public files
│   ├── _src/                 source files
│   ├── README.txt            public report
│   ├── rv-101-filename1.py    public rivt file
│   ├── rv-102-filename2.py    public rivt file
│   ├── rv-201-filename3.py    public rivt file
│   └── rv-202-filename4.py    public rivt file
├── ...
├── [rivt-report]/            rivt files and docs
│   ├── [rv101-]filename1.py    rivt file
│   ├── [rv102-]filename2.py    rivt file
│   ├── [rv201-]filename3.py    rivt file
│   └── [rv202-]filename4.py    rivt file
├── ...
├── [src_]                    author source files
│   ├── data/                  tables
│   │   ├── opensees1.txt
│   │   └── conc-vals.csv
│   ├── image/
│   │   ├── fig1.png
│   │   └── fig2.jpg
│   ├── run/                   OS commands
│   │   ├── run1_win.cmd
│   │   └── run1_linux.sh
│   ├── tools/                 scripts and functions
│   │   ├── coverpage.rst
│   │   ├── logoname.png
│   │   ├── plot.py
│   │   └── loads.py
│   ├── vals/                  value files
│   │   ├── steel-vals.csv
│   │   └── plastic-vals.csv
├── [_published/              published docs and reports
│   ├── [docs]/                html docs
│   │   ├── _downloads/
│   │   ├── _images/
│   │   ├── _locale/
│   │   ├── _sources/
│   │   ├── _sphinx_design_static/
│   │   ├── _static/
│   │   ├── _templates/
│   │   ├── .doctrees/
│   │   ├── .buildinfo
│   │   ├── .buildinfo.bak
│   │   ├── .nojekyll
│   │   ├── genindex.html
│   │   ├── search.html
│   │   ├── objects.inv
│   │   ├── searchindex.js
│   │   ├── rv101-filename1.html
│   │   └── rv102-filename2.html

```



[2] Folder Names

Reports are organized using the following folders with the required names and prefixes shown in italics:

rivt-report-label

Top level rivt folder containing rivt report and public files.

rivt-public_

Includes *public rivt files* written by *rivtlib* for public sharing. The prefix *rvAnn-* is changed to *rv-Ann-* to avoid confusion with private files.

rivt-report

Includes folders for source and published files and *docs*.

_src

Includes source files used by *rivt* files including data, images, run commands, tools and value files.

published_

Includes formatted *docs* and *reports* .

stored_

Includes output files written by *rivtlib* including *logs*, *values*, *hidden sections* and *metadata*.

rstdocs_

Includes intermediate restructured text files written by *rivtlib*.

C.4 Reports

[1] Report compilation

A typical workflow for writing *reports* is to start with a similar *rivt* folder and edit the *rivt* files and *report script* that produces the *report*.

A report is assembled from multiple *docs* organized by division and subdivision numbers provided in the *rivt* file name. Each *doc* is a subdivision of a *report*. A *rivt* file number has the form:

```
rvAnn-filename.py
```

where *rv* is a required prefix, *A* is an alphanumeric character and *nn* are two digit non-negative integers. The prefix is used to organize reports into divisions and subdivisions.

Corresponding *rivt* docs are output as:

```
rvAnn-filename.txt  
rvAnn-filename.pdf  
rvAnn-filename.html
```

A *rivt* report is organized using the *rivt* file numbers. If the *rivt* file names are:

```
rvA01-filename.py  
rv105-filename.py  
rv212-filename.py
```

the corresponding *doc* numbers in the would be:

- A.1 (division A, subdivision 1)
- 1.5 (division 1, subdivision 5)
- 2.12 (division 2, subdivision 12)

Note that leading zeroes are dropped in the formatted output.

[2] rivt-report.py

The Python *report script* includes settings that specify assembly parameters and override defaults. These include:

- Docs to be included in the report
- Division names
- recompiling docs
- report cover page and title

provides the option to either regenerate all docs or to assemble the report from previously generated docs. Most aspects of the report appearance are determined at when generating docs.

An example rivt-report.py script is shown below.

```
#!/ python

from rivtlib.rvreport import * # noqa: F403

""" generate a rivt report

Sample report generating and config file.

Run this Python file to generate a rivt report. The report generating file
must located in /publish/subfolder where subfolder is the doc type. The
report output file name is the report title. It is written to the same
folder as the report script. Duplicate report file names are incremented.

A flag determines whether the report is assembled from existing doc files
or whether docs are regenerated prior to assembly. If a rivt or doc file is
specified for inclusion and not found a warning is given but the report is
still assembled.

A rivt report is a compilation of rivt docs organized into divisions and
subdivisions. rivt doc numbers define the organization. Default titles for
subdivisions are generated by stripping the doc number from the file name
and replacing hyphens with spaces. The default titles may be overridden in
rv.M for each rivt file.

See src/styles/rivt.ini for additional settings

"""
# =====
# report type
# =====
rptype = "pdf" # report type [html; pdf; pdftex; text]
rvrun = False # regenerate docs [True; False]

# =====
# report cover settings
# =====
# cover pages are located in /src
rptitle = "Solar Canopy Calculations"
rpstitle = "Larkspur, Ca"
rpauthor = "rhh"
rpdate = "<datetime>"
rptoc = True # add table of contents, "" to omit
rpcover = "/publish/cover1.pdf" # cover page, "" to omit
```

```

# =====
# include docs
# =====
# include these divisions in report
divlist = [[dv01, "Codes and Loads"],
            [dv02, "Frame"],
            [dv03, "Foundation"]]
# include these docs in report
# doc label is for reference only, may be empty string
doclist = [[rv101, "Codes"],
            [rv102, "Loads"],
            [rv201, "Frame"],
            [rv202, "Solar"],
            [rv301, "Foundation"],
            [rv302, "Walls"]]

genreport() # noqa: F405

```

[3] conf.py

The *conf.py* file contains configuration settings for *rivt docs* and reports. A typical *conf.py* file looks like this:

```

#!/ python

from rivtlib.rvconfig import * # noqa: F403

""" configuration settings for rivt docs and reports

This file is stored in /src/Tools. Settings for both HTML and PDF docs are
included and cover:

- metadata
- formatting

"""

import sys
import os
from pathlib import Path

sys.path.append(str(Path(".").resolve()))

project = "rivt doc"
copyright = "2023 StructureLabs LLC"
author = "rholland"
release = "0.3.8a7"

#
# -- Options for HTML and PDF output -----
#

```

```

extensions = [
    "sphinx.ext.githubpages",
    "sphinx_togglebutton",
    "sphinxcontrib.jquery",
    "sphinx_copybutton",
    "sphinx_favicon",
    "sphinx.ext.duration",
    "sphinx.ext.doctest",
    "sphinx.ext.autodoc",
    "sphinx_design",
    "sphinx_new_tab_link",
    "rst2pdf.pdfbuilder",
]
root_doc = "index"
duration_write_json = ""
html_show_sourcelink = False
exclude_patterns = ["_build", "Thumbs.db", ".DS_Store"]
source_suffix = [".rst", ".md"]
templates_path = ["_templates"]
locale_dirs = ["_locale"]
html_title = "rivt"
html_theme = "pydata_sphinx_theme"
html_context = {"default_mode": "dark"}
html_sidebars = {"**": ["sidebar-nav-bs.html"]}
html_static_path = ["_static", "_static/img"]
html_css_files = ["css/custom.css"]
html_theme_options = {
    "pygments_light_style": "tango",
    "pygments_dark_style": "github-dark",
    "navbar_start": ["navbar-logo"],
    "collapse_navigation": True,
    "header_links_before_dropdown": 6,
    "navbar_align": "left",
    "show_toc_level": 1,
    "navigation_depth": 1,
    "footer_start": ["copyright"],
    "footer_end": [],
    "logo": {
        "text": "rivt",
        "image_dark": "rivhomellc.png",
        "image_light": "rivhomellc.png",
    },
}
}
favicons = [
    {
        "rel": "icon",
        "sizes": "16x16",
        "href": "favicon-16x16.png",
    },
    {
        "rel": "icon",

```

```

        "sizes": "32x32",
        "href": "favicon-32x32.png",
    },
]

#
# -- Options for PDF output -----
#
# source start file, target name, title, author, options
# options: ('index', 'MyProject', 'My Project', 'Author Name', {'pdf_compressed': True})
# More than one author : \\r'Guido van Rossum\\Fred L. Drake, Jr., editor'
pdf_documents = [("rv001-single-doc", "rv001-single-doc", "rivt", "r holland")]
# Label to use as a prefix for the subtitle on the cover page
subtitle_prefix = "User Manual"
# A list of folders to search for stylesheets. Example:
pdf_style_path = [".\\rstdocs/_static/pdfstyle"]
# A colon-separated list of folders to search for fonts. Example:
pdf_font_path = [".\\rstdocs/_static/fonts"]
# A comma-separated list of custom stylesheets. Example:
pdf_stylesheets = [".\\rstdocs/_static/pdfstyle/stylepdf1.yaml"]
# Example: compressed=True
pdf_compressed = False
# Language to be used for hyphenation support
pdf_language = "en_US"
# literal blocks wider than the frame overflow, shrink or truncate
pdf_fit_mode = "shrink"
# 1 means top-level sections start in a new page 0 disabled
pdf_break_level = 0
# When a section starts in a new page, force it to be 'even', 'odd',
# or just use 'any'
pdf_breakside = "any"
# If false, no coverpage is generated.
pdf_use_coverpage = True
# Name of the cover page template to use
pdf_cover_template = "pdfcover.rst"
# Page template name for "regular" pages
# pdf_page_template = 'cutePage'
# Documents to append as an appendix to all manuals.
# pdf_appendices = []
# Enable experimental feature to split table cells. Use it
# if you get "DelayedTable too big" errors
# pdfSplittables = False
# Set the default DPI for images
# pdf_default_dpi = 72
# Enable rst2pdf extension modules
# pdf_extensions = []
# Show Table Of Contents at the beginning?
pdf_use_toc = True
# How many levels deep should the table of contents be?
pdf_toc_depth = 9999
# Insert footnotes where they are defined instead of
# at the end.

```

```

pdf_inline_footnotes = True
# verbosity level. 0 1 or 2
# pdf_verbosity = 0
# If false, no index is generated.
pdf_use_index = True
# If false, no modindex is generated.
pdf_use_modindex = False
# Add section number to section references
pdf_use_numbered_links = False
# Background images fitting mode
pdf_fit_background_mode = "scale"
# Repeat table header on tables that cross a page boundary?
pdf_repeat_table_rows = True
# Enable smart quotes (1, 2 or 3) or disable by setting to 0
pdf_smartquotes = 0

```

C.5 Example

[1] rivt file

This is a small rivt file that publishes a single *doc*. It has 4 API functions - two of them are published. The output is written to the terminal.

```

#!/ python
import rivtlib.rvapi as rv

# rv singledoc: True

# %% loads
rv.I("""Load Combinations

ASCE 7-05 Load Effects _[T]
=====
Equation No.    Load Combination
=====
16-1            1.4(D+F)
16-2            1.2(D+F+T) + 1.6(L+H) + 0.5(Lr or S or R)
16-3            1.2(D+F+T) + 1.6(Lr or S or R) + (f1L or 0.8W)
=====

| IMAGE | beam.png | Beam Geometry, .65, num

Bending Stress Formula _[E]
σ1 = M1 / S1 _[M]

""")
# %% values
rv.V("""Loads and Geometry

Beam Loads _[T]
D_1 =: 3.8*psf | psf, kPA, 2 | joists DL

```

```

D_2 =: 2.1*psf | psf, kPA, 2 | plywood DL
D_3 =: 10.0*psf | psf, kPA, 2 | partitions DL
D_4 =: 2*0.5*klf | klf, kN_m , 2 | fixed machinery DL
L_1 =: 40*psf | psf, kPA, 2 | ASCE7-05 LL

| VALTABLE | beam1-v.csv | Beam Geometry, 0:0, num

Uniform Distributed Loads
dl_1 <=: 1.2 * (W_1 * (D_1 + D_2 + D_3) + D_4) | klf, kN_m, 2 | dead load : ASCE7-05 2.3.2 _[E]

ll_1 <=: 1.6 * W_1 * L_1 | klf, kN_m, 2 | live load : ASCE7-05 2.3.2 _[E]

omega_1 <=: dl_1 + ll_1 | klf, kN_m, 2 | total load : ASCE7-05 2.3.2 _[E]

    """)
# %% section properties
rv.V("""Beam Section Properties

| PYTHON | sectprop.py | nodocstring

section_1 <=: rectsect(10*inch, 18*inch) | in3, cm3, 2 | function: rect sect modulus _[E]

inertia_1 <=: rectinertia(10*inch, 18*inch) | in4, cm4, 1 | function: rect moment inertia _[E]

    """)
# %% force
rv.V("""Force and Stress

    m_1 <=: omega_1 * S_1**2 / 8 | ftkips, mkN, 2 | mid-span UDL moment _[E]

    fb_1 <=: m_1 / section_1 | psi, MPA, 1 | bending stress _[E]

    fb_1 < 10*ksi | ok, not ok, blue, red, right _[E]

    """)
# %% tool
rv.T("""Metadata

    _[[PYTHON]]
    rv_metaD = {
    "authors": "rholland",
    "version": "0.7.2",
    "email": "rod.h.holland@gmail.com",
    "repo": "https://github.com/rivt-info/rivt-single-doc",
    "license": "https://opensource.org/licenses/mit/",
    "fork1": ["author", "version", "email", "repo"],
    "fork2": [],
    }
    _[[END]]

    """)
# %% doc

```

```
rv.D("""Publish Doc
```

```

    _[[LAYOUT]]
    logopathP: logo.png
    pdfheaderL: [docnameS, pageS, totalpageS]
    footerL: [logo0, date0, time0, filenameS, versionS]
    _[[END]]

```

```
| PUBLISH | rivt | rst2pdf
""")
```

D.1 Markup

[1] API functions

rivt has seven API functions. The name *rivt* is an acronym taken from the four functions that process content. The remaining three functions are used for document generation and debugging.

API Function	Name	Purpose
rv.R (rS)	Run	Run scripts and markup
rv.I (rS)	Insert	Insert static sources
rv.V (rS)	Values	Calculate values
rv.T (rS)	Tools	Execute shell scripts and external programs
rv.D (rS)	Doc	Publish docs
rv.S (rS)	Skip	Skip section (comments and debugging)
rv.X (rS)	Exit	Exit rivt (debugging)

API functions define doc sections. If interactive IDEs like *VSCode* or *Spyder* are used to edit and run *rivt* files, functions can be processed individually as cells by using the standard notebook percent notation:

```
# %% optional label
rv.API("""rivt string""")
```

[2] rivt string

Each API function takes a triple quoted rivt string argument composed of a header substring line followed by a multiple-line content substring. The *header substring* defines section processing parameters. The *content substring* includes rivt markup and is indented four spaces for improved readability and navigation (e.g. section folding).

Header substring

The header substring starts with a *section label*, used as the section title, followed by a vertical bar and three comma separated *section parameters* that override default behavior.

```
rv._("""Section Label | doc;stored, private;public, section;merge
    Content substring indented 4 spaces
    ...
```

```
""")
```

The header parameters include the following comma separated values in any order:

private/public

Determines whether the API section text is copied to the the *Public* folder *rvt* file for sharing.

doc/stored

Determines whether the *rvt string* is formatted and printed in the doc, or just annotated in the doc and written to a file in the *Stored* folder for optional inclusion as an appendix.

section/merge

Determines whether the API starts a new *doc* section or is merged into the previous section.

Default settings in the *header substring* do not need to be specified. The default setting for each API is listed first (in bold) in the table below. The default privacy settings for all sections in a *rvt* file may be reversed by including the *public comment setting* immediately following the *rvtlib import statement*.

```
import rivtlib.rvapi as rv
# rv setpublic = true
```

Individual sections may still be set as private in the header substring.

API	private; public	doc; stored	section; merge
rv.R	private ; public	stored ; doc	merge ; section
rv.I	private ; public	doc ; stored	section ; merge
rv.V	private ; public	doc ; stored	section ; merge
rv.T	private ; public	stored ; doc	merge ; section
rv.S	private ; public	stored	merge
rv.D	public	stored	merge
rv.X	•	•	•

Examples of *header substring* settings are shown below.

- An example with explicit defaults (they do not have to be declared).

```
# This
rv.I("""A New Section | private, doc, section
    Content text
    ...
```

[3] Tags and Commands

```
    """)
# is equivalent to:
rv.I("""A New Section
    Content text
    ...
    """)
```

- An example that merges a section into the previous section.

```
rv.I("""A Merged Section | merge
    Content text
    ...
    """)
```

Content substrings

The content substring is indented four spaces for legibility and code folding. It includes line tags, block tags and commands along with text.

```
rv._("""Section Label
    Content text indented 4 spaces.
    ...
    """)
```

Content is converted line by line into formatted text and [RestructuredText](#), and then further processed into HTML or PDF. If a line does not contain a *command* or *tag* it is passed through as is, which allows for the *Insert* function (rv.I) including some *restructured text* directly i.e. surrounding words with *** or **** will format a word as italic or bold respectively.

In addition block tags in the *Tools function* (rv.T) directly supports processing HTML, LaTeX and reStructuredText scripts.

[3] Tags and Commands

Line Tags

A line tag formats a line of text and is denoted with a single **_[LETTER]**, placed at or near the end of the line, depending on the tag.

Block Tags

[4] Folders

A block tag formats a block of text and begins with `_[[TAGNAME]]` and terminates with `_[[END]]`.

Commands

rivt commands read and write external files. They typically start in the first column with a vertical bar (|) followed by the command name, file path, and parameters.

The exceptions are the definition (`==:`), assignment (`<=:`), function (`:=:`) and compare (`<>`) commands that are used to define, assign and compare values.

```
| COMMAND | relative path | parameters
```

File paths are specified relative to the *rivt root folder*. The *rivt report* folder structure is described here.

If the path is omitted the default path for each command is applied. If the *singledoc* parameter is set, the *resource files* and *docs* are stored in the *rivt root folder*.

Tag and Command syntax for each API type is defined and described using the following format:

[4] Folders

The top level folder structure is shown below. A more detailed description of the folder structure is here.

Top Level Folders

[rivt-]Report-Label/	Report Folder
├─ [_public]/	rivt-generated public files
│ └─ [src_]/	source files
│ └─ README.txt	rivt-generated public report
│ └─ rv-101-filename1.py	public rivt file
│ └─ rv-102-filename2.py	public rivt file
│ └─ rv-201-filename3.py	public rivt file
│ └─ rv-202-filename4.py	public rivt file
└─ ...	
└─ [rivt-report]/	rivt files and docs
│ └─ .vscode/	optional VSCode settings
│ └─ [_published]/	published docs and reports
│ └─ [_rstdocs]/	restructured text files
│ └─ [_stored]/	stored files
│ └─ [src_]/	source files
│ └─ [config-report.py]	report generating script
│ └─ README.txt	rivt-generated report
│ └─ [rv101-]filename1.py	rivt file
│ └─ [rv102-]filename2.py	rivt file
│ └─ [rv201-]filename3.py	rivt file
│ └─ [rv202-]filename4.py	rivt file
└─ ...	

D.2 Line Tags

[1] Line Tag Summary

Format a line of text

API Scope	Line Tag	Description
rv.l,V	text _ [C]	[2] Center text
rv.l,V	text _ [R]	[3] Right justify text
rv.l,V	text math _ [M]	[4] Text math
rv.l,V	LaTeX math _ [L]	[5] LaTeX math
rv.l,V	text _ [E]	[12] Equation number
rv.l,V	title _ []	[13] Table number
rv.l,V	caption _ [F]	[14] Figure number
rv.l,V	text _ [#] text	[6] Endnote number
rv.l,V	text _ [G] glossary term text	[7] Glossary link
rv.l,V	text _ [S] label, section text	[8] Section link
rv.l,V	text _ [U] label, url text	[10] URL link
rv.l,V	text _ [V] var_name text	[11] Variable value
all	## text	nonprinting comment

[2] Center text

Center line of text within the page margins.

_[C]

Syntax:

text **_**[C]

Example:

This text will be centered. **_**[C]

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[4] Folders

[3] Right justify text

Right justify line of text within the page margins.

`_[R]`

Syntax:

`text _[R]`

Example:

`This text wil be right justified _[R]`

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[4] Text math

Format math expression into text.

`_[M]`

Syntax:

`text math expression _[M]`

Example:

`f(x,y) = sin(x)**2 + y/5 _[M]`

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[5] LaTeX math

Format LaTeX math expression.

`_[L]`

Syntax:

`LaTeX math expression _[L]`

Example:

`\frac{1}{\sqrt{x}} _[L]`

[4] Folders

API Scope	Insert, Values
Doc Types	PDF, HTML

[6] Endnote number

Assign endnote number to the text in order of processing. Endnotes are defined with the block tag `_[[ENDNOTE]]` and are listed at the end of the *doc*.

`_[#]`

Syntax:

text `_[#]` more text

Example:

This is a sentence with an endnote `_[#]` tag.

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[7] Glossary link

Link a term to the glossary.

`_[G]`

Syntax:

text `_[G]` glossary term

Example:

A definition of `_[G]` namespace
is provided in the glossary.

API Scope	Insert, Values
Doc Types	PDF, HTML

[8] Section link

Create a link to the section label defined in the API header with optional link text. If the text is omitted the section label is used for the link.

[4] Folders

`_ [S]`

Syntax:

text `_ [S]` link text, section title

Example:

This creates a link to `_ [S]` Section Title, actual section label which provides more detail.

API Scope	Insert, Values
Doc Types	PDF, HTML

[9] Doc link

Link to a *doc* in a *report* with an optional link text. If the link text is omitted the *doc* title will be inserted as the link term.

`_ [D]`

Syntax:

text `_ [D]` text, rivt_file

Example:

This is a link to `_ [D]` rv101-filename1
for reference.

API Scope	Insert, Values
Doc Types	PDF, HTML

[10] URL link

Link to a an external site with optional link text. If the link text is omitted the url will be inserted as the link term.

`_ [U]`

Syntax:

text `_ [U]` link label ,external url

Example:

text `_ [U]` github, <https://www.github.com>

[4] Folders

API Scope	Insert, Values
Doc Types	PDF, HTML

[11] Variable value

Insert the value of var_name `_V` in the sentence.

`_V`

Syntax:
text var_name `_V` more text

Example:
The value of my_var is my_var `_V`.

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[12] Equation number

Assign equation number to a line of text.

`_E`

Syntax:
math text or label `_E`

assignment command `_E`

Example:
 $3x * 4/(1+n)$ `_E`

$x \leq 3*IN + 4.1*FT$ | inch, cm, 2 | example 1 `_E`

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[13] Table number

_ [T]

Syntax:
 Table Title _ [T]
 Example:
 A New Table _ [T]

API Scope	Insert, Values
Doc Types	text, PDF, HTML

[14] Figure number

_ [F]

Syntax:
 Figure caption _ [F]
 Example:
 Stress Distribution _ [F]

API Scope	Insert, Values
Doc Types	text, PDF, HTML

D.3 Block Tags

[1] Block Tag Summary

Format blocks of text or run code

API Scope	Block Tag	Description
rv.R	_ [[SHELL]] <i>os, wait;nowait</i>	[2] Shell script
rv.l	_ [[BOX]] label	[8] Box
rv.l	_ [[TOPIC]] topic	[7] Topic
rv.V	_ [[TABLE]] label	[6] Text
rv.T	_ [[MARKUP]] type	[10] Markup block

API Scope	Block Tag	Description
rv.D	<code>__[[METADATA]]</code> label	[11] Metadata
all	<code>__[[END]]</code>	[13] End block

[2] Shell script

Runs shell scripts that run external programs. The shell parameters include specifying the operating system, process control and terminal window control. The `os` parameter specifies the terminal type. The `wait` parameter specifies whether rvt file processing waits for the script to complete before continuing. The `open` parameter specifies whether to keep the shell window open after execution.

`__[[SHELL]]`

Syntax:

```
__[[SHELL]] win;mac;linux,wait; nowait,open; close
shell command
...
__[[END]]
```

Example:

```
__[[SHELL]] win,nowait,open
dir
path
__[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[3] Indent text

Indents text block the specified number of spaces.

[4] Folders

`_[[INDENT]]`

Syntax:

```
_[[INDENT]] number of spaces
text
text
...
_[[END]]
```

Example:

```
_[[INDENT]] 8
This is a sentence that will be
indented 8 spaces.
_[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[4] Indent italic

`_[[ITALIC]]`

Italicizes and indents the text block the specified number of spaces.

Syntax:

```
_[[ITALIC]] number of spaces
text
...
_[[END]]
```

Example:

```
_[[ITALIC]] 4
This is a sentence that will be
italicized and indented 4 spaces.
_[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[5] Endnotes

`_[[ENDNOTES]]`

Assigns numbers and formats endnotes in order of processing. Each endnote is separated by a blank line and is numbered in order of occurrence.

Syntax:

```
_[[ENDNOTES]] label
Endnote 1.
```

```
Endnote 2.
```

```
...
_[[END]]
```

Example:

```
_[[ENDNOTES]] ACI citations
```

```
This endnote is assigned to the first endnote tag (_[#]) in order of
of processing.
```

```
This second endnote is separated by a blank line and assigned
to the second
```

```
This is a third endnote.
_[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[6] Text

```
_[[TABLE]]
```

Reads and formats restructured text table and writes contents to csv file in the _stored/Vals folder. ..
code-block:: text

Syntax:

```
_[[TABLE]] label reST table ... _[[END]]
```

Example:

```
_[[TABLE]] my_table ===== Name Age ===== John 25 Jane 30 ===== _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[6] Text

```
_[[TEXT]]
```

Reads and formats text and code. The language parameter specifies formatting and syntax coloring. Languages include:

- *literal*

[4] Folders

- *python*
- *text*
- *sh*
- *cmd*

Syntax:

```
_[[TEXT]] language  
text and code  
...  
_[[END]]
```

Example:

```
_[[TEXT]] python  
# some code  
print(1)  
a = 1 + 3  
_[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[7] Topic

_[[TOPIC]]

Formats a topic block.

Syntax:

```
_[[TOPIC]] topic title  
text  
...  
_[[END]]
```

Example:

```
_[[TOPIC]] topic title  
Text related to the topic.  
_[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[8] Box

`_[[BOX]]`

Draws a box around the block of text.

```
Syntax:
  _[[BOX]] optional label
  text
  ...
  _[[END]]

Example:
  _[[BOX]]
  This is a sentence.
  A second sentence.
  A box is drawn around the sentences.
  _[[END]]
```

API Scope	Insert
Doc Types	text, PDF, HTML

[9] Python

`_[[PYTHON]]`

Executes Python script in the *rivt namespace* or a user specified namespace. File paths in the script are relative to the *rivt file* folder.

```
Syntax:
  _[[Python]] topic title
  code
  ...
  _[[END]]

Example:
  _[[PYTHON]] *rvspace*; user namespace
  b = 1*inch + 2.2*ft
  print(b)
  _[[END]]
```

API Scope	Value, Tool
Doc Types	text, PDF, HTML

[10] Markup block

Inserts HTML into an HTML *doc*, LaTeX into a PDF *doc*, and reStructuredText into either PDF or HTML.

`_[[MARKUP]]`

```
Syntax:
_[[MARKUP]] *html;latex;rst*
markup
...
_[[END]]
```

```
Example:
_[[MARKUP]] rst
This is bold and
this is italic in
reStructuredtext
...
_[[END]]
```

API Scope	Tool
Doc Types	text, PDF, HTML

[11] Metadata

`_[[METADATA]]`

Metadata is written to the *rvAnn-apilog.py* file and is converted internally to a Python dictionary *rvmetaD* with key name taken from the label e.g. *docnameS* or *authorsD*.

```
Syntax - defaults:
_[[METADATA]] optional label
docname = ""
authors = ""
version = 0.0.0
email = ""
repo = ""
license = https://opensource.org/license/mit
fork1 = author, version, email, repo
fork2 = author, version, email, repo
_[[END]]
```

```
Example:
_[[METADATA]]
docname = Bearing Pressures
authors = rholland, rward
version = 0.1.1
email = rholland@email.com
```

[4] Folders

```
repo = https://github.com/rivt-info/rivt-simple-doc
license = https://opensource.org/license/mit
_{{END}}
```

API Scope	Docs
Doc Types	text, PDF, HTML

[12] Layout block

Overrides default layout settings.

_{{LAYOUT}}

```
Syntax - defaults:
  _{{LAYOUT}} optional label
  variables
  ...
  _{{END}}

Example:
  _{{LAYOUT}}
  docname = ""
  footer = date, docname, page
  _{{END}}
```

API Scope	Doc
Doc Types	text, PDF, HTML

[13] End block

_{{END}}

Terminates a block tag.

```
Syntax:
  _{{NEWPAGE}} optional label

Example:
  _{{END}}
```

API Scope	All
Doc Types	text, PDF, HTML

D.4 Commands

[1] Command Summary

format files and equations

API Scope	Command	Description
rv.R	SHELL rel path os, wait	[2] Shell file
rv.V, I	TABLE rel path title,width,rows,align,head,num;non	[4] Table file
rv.V, I	IMAGE rel path caption, scale, num;non	[5] Image file
rv.V, I	IMAGE2 rel pth1, rel pth2 ca1,ca2,sc1,sc2,num;non	[6] Adjacent images
rv.V	PYTHON rel path rvt;namespace	[11] Python file
rv.V	VALTABLE rel path title, rows, num;non	[7] Valtable file
rv.V	a ==: 1*IN unit1, unit2, decimal label	[8] Define value
rv.V	c <=: expression unit1, unit2, decimal label	[9] Assign value
rv.V	c :=: func(x,y) unit1, unit2, decimal label	[9] Function value
rv.V	a < c unit, decimal, text1, text2 label	[10] Compare values
rv.T	MARKUP rel path type	[12] Markup file
rv.D	ATTACHPDF rel path place, title	[13] Attach PDF
rv.D	PUBLISH doc title type	[14] Publish doc

Default command paths

See here for the folder structure. If files are in the default path only the file name needs to be provided.

Command	Default Path	R/W
SHELL	/_src/	R
TABLE	/_src/	R
IMAGE	/_src/	R
IMAGE2	/_src/	R
VALTABLE	/_src/ [1]	R
PYTHON	/_src/	R
MARKUP	/_src/	R

Command	Default Path	R/W
ATTACHPDF	<i>/_src/</i>	W
PUBLISH	<i>/published_/</i> [2]	W

[1] use */_stored/vals/* to read values defined in the rvt file [2] files are written to the type subdirectory

[2] Shell file

The SHELL command runs shell scripts including .cmd, .bat and .sh files. The *os* parameter specifies the operating system: *win*, *mac* or *linux*. The *wait*; *nowait* specifies whether rvt file processing waits for the script to complete before continuing.

If the *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/run/*. Otherwise the path is specified relative to the report root (rvt file folder). If the *doc* is a single doc the file is read from the rvt file folder.

Syntax:

```
| SHELL | relative file path | win;mac;linux,wait;nowait
```

Example:

```
| SHELL | file1.cmd | win, nowait
```

API Scope	rv.R
File Types	.cmd, .bat, .sh, .bsh
Doc Types	text, PDF, HTML

[3] Text file

The TEXT command reads and formats text and code files. The *language* parameter specifies formatting and syntax coloring. Language types include:

- *literal*
- *python*
- *text*
- *sh*
- *cmd*
- *reStructuredText*

The *literal* type inserts text into the *doc* without formatting.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/data/*. Otherwise the path needs to be specified relative to the report root (rvt file folder). If the *doc* is a single doc the file is read from the rvt file folder.

Syntax:

```
| TEXT | relative file path | language - see list above
```

Example:

```
| TEXT | file1.txt | literal
```

API Scope	rv.l
File Types	txt, .py, .cmd, .bat, .sh, .rst
Doc Types	text, PDF, HTML

[4] Table file

The TABLE command reads csv, xls, and rst files and outputs formatted tables. The title may be omitted by inserting a hyphen "-". The width parameter specifies the maximum character width of a column. The align parameter specifies the cell justification - left, center, right. The number parameter specifies whether the table is numbered. For csv files, the head parameter specifies whether the first row is a column header.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/data/*. Otherwise the path needs to be specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

| TABLE |

Syntax:

```
| TABLE | rel path | title, max width, rows, l;c;r, head;nohead, num;nonum
```

Example:

```
| TABLE | file1.csv | Forces, 30, 0:0, c, nohead, num
```

API Scope	rv.l, rv.V
File Types	csv, xls, rst
Doc Types	text, PDF, HTML

[5] Image file

The IMAGE command reads a PNG or JPEG file and centers it in the *doc*. The scale parameter is a decimal fraction of the page width. The caption may be omitted by using a single hyphen. The *num;nonum* parameter specifies whether to assign a figure number. The image path is inserted in the text *doc* instead of the image.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/img/*. Otherwise the path needs to be specified relative to the report root (rivt file folder). If the doc is a single doc the file is read from the rivt file folder.

Syntax:

```
| IMAGE | relative path | caption, scale, num;nonum
```

Example:

```
| IMAGE | file1.png | Map, 0.5, num
```

API Scope	rv.V, rv.I
File Types	PNG, JPG
Doc Types	PDF, HTML

[6] Adjacent images

The IMAGE2 command reads two PNG or JPEG file and places them side by side in the *doc*. The scale parameters are a decimal fraction of the page width. The captions may be omitted by using a single hyphen for either or both images. The *fig;nofig* parameters specify whether to assign a figure number to either or both images. The image path is inserted in the text *doc* instead of the image.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/img/*. Otherwise the path needs to be specified relative to the report root (rivt file folder). If the *doc* is a single *doc* the file is read from the rivt file folder.

Syntax:

```
| IMAGE2 | relative path | cap1, cap2, scale1, scale2, num;nonum, num;nonum
```

Example:

```
| IMAGE2 | file1.png, file2.png | Map, Photo, .5,.5, num, num
```

API Scope	rv.I, rv.V
File Types	PNG, JPG jenn
Doc Types	PDF, HTML

[7] Valtable file

The VALTABLE command imports and defines values from a *csv* or *xls* file. The file format is:

```
var = value, unit1, unit2, decimal, label
```

The path variable is either from *src* or *stored* depending on if the values were generated by rivtlib or provided by the author. The title parameter is the table title. A hyphen omits the title. The *rows* parameter specifies the rows to import. The *num; nonum* parameter specifies whether to assign a table number to the values table.

If a *doc* is part of a report and no path is specified, the file is assumed to be in the default folder */src/vals/*. Otherwise the path needs to be specified relative to the report root (rivt file folder). If the values are read from prior calculated values, they will be found in the */stored/vals* folder. If the *doc* is a single *doc* the file is read from the rivt file folder.

Syntax:

```
| VALTABLE | relative path | title, rows, *num;nonum*
```

Example:

If read from the default folder:

```
| VALTABLE | newvals.csv | Beam Properties, 1:10, num
```

If read from the stored folder:

```
| VALTABLE | /stored/vals/vA01-2.csv | Beam Properties, 1-10, num
```

API Scope	rv.V
File Types	.csv, .xls,
Doc Types	text, PDF, HTML

[8] Define value

Defines a value and writes it to the file *vdocnum-s.csv* where *num* is the *docnumber* and *s* is the section number. The file is written to the folder *stored/vals* unless *singledocB* is set to *True* in the comment variable.

The stored values can read and defined in other rivt files using the VALUES command.

Syntax:

```
c =: 5*unit | unit1, unit2, decimals | label, *num,nonum*
```

Example:

```
D_1 =: 10*IN | IN, M, 3 | beam depth, num
```

API Scope	rv.V
File Types	.csv
Doc Types	text, PDF, HTML

[9] Assign value

Assigns a value to an equation and writes the values to a file *vdocnum-s.csv* where *num* is the *doc number* and *s* is the section number. The file is written to the folder *stored/vals* unless *rv single doc* is set to *True* in which case values are stored in the rivt file folder (root).

The label is a reference printed with the equation. The units specify the result expressed in two different units. The integer specifies the number of places after the decimals.

Syntax:

```
b <=: a * 10*FT | unit1, unit2, decimals | label, num;nonum
```

Example:

```
b_1 <=: E_1 * 12.1*IN^2 | KIP, KN, 2 | Std. 123, num
```

API Scope	rv.V
File Types	.csv
Doc Types	text, PDF, HTML

[9] Function value

Assigns a value to an equation and writes the values to a file *vdocnum-s.csv* where *num* is the *doc number* and *s* is the section number. The file is written to the folder *stored/vals* unless *rv single doc* is set to *True* in which case values are stored in the rivt file folder (root).

The label is a reference printed with the equation. The units specify the result expressed in two different units. The integer specifies the number of places after the decimals.

Syntax:

```
c ::= function name() | unit1, unit2, decimals | label, num;nonum
```

Example:

```
c_1 ::= func1(a,b) | KIP, KN, 2 | ACI 318-19 Table 22.5.5.1, num
```

API Scope	rv.V
File Types	.csv
Doc Types	text, PDF, HTML

[10] Compare values

Inserts a text with alignment and color on the following line if the expression evaluates to true. text1 and text2 are printed if true or false respectively. Colors are assigned accordingly as *red;blue;yellow;green;black;gray* defined in the *style files*. The text is right aligned.

Comparison operators:

```
==    Equal x == y
!=    Not equal    x != y
>     Greater than x > y
<     Less than    x < y
>=    Greater than or equal to x >= y
<=    Less than or equal to x <= y
```

<>

Syntax:
`a < b | text1, text2, color1, color2`

Example:
`a < b | OK, NOT OK, blue, red`

API Scope	rv.V
Doc Types	text, PDF, HTML

[11] Python file

Executes Python code in the *rv namespace* or user specified namespace. File paths used in the script are relative to the root *rv* folder.

Syntax:
`| PYTHON | relative path | *rvspace*; user space`

Example:
`| PYTHON | script1.py | rv-space`

API Scope	rv.V, rv.T
File Types	.csv
Doc Types	text, PDF, HTML

[12] Markup file

Inserts HTML into an HTML *doc*, LaTeX into a PDF *doc*, and reStructuredText into either PDF or HTML. LaTeX requires the installation of *texlive*.

| MARKUP |

Syntax:
`| MARKUP | relative path | *rst, html, latex*`

Example:
`| MARKUP | script.rst | rst`

API Scope	rv.t
File Types	html, rst, tex

[4] Folders

API Scope	rv.t
Doc Types	text, PDF, HTML

[13] Attach PDF

Appends or prepends a PDF file to the *doc*. The title parameter generates an Appendix cover page with the specified title. A "-" omits the over page. For HTML *docs* the file is inserted as a download link.

| ATTACHPDF |

Syntax:

| ATTACHPDF | relative path | *front;back*, title

Example:

| ATTACHPDF | relative path | *front;back*, title

API Scope	rv.D
File Types	tex, html, text, pdf
Doc Types	text, PDF, HTML

[14] Publish doc

Publishes a *doc*.

Syntax:

| PUBLISH | ini relative path | rst2pdf, texpdf, tex, hmtl

Example:

| PUBLISH | ini relative path | rst2pdf

API Scope	rv.D
File Types	tex, html, text, pdf
Doc Types	text, PDF, HTML

D.5 Run - rv.R

[1i] rv.R Markup

The *Run* API function executes shell commands.

[2i] Block Tags

Block Tag	Description
<code>_[[SHELL]]</code> process parameters	[2] Shell script
<code>_[[END]]</code>	[13] End block

[3i] Commands

Command	Description
SHELL relative path os, wait	[2] Shell file

D.6 Insert - rv.l**[1i]** rv.l Markup

The *Insert* API function inserts and formats static sources into the *doc*, including images, tables, links and formatted text.

[2i] Line Tags**[3i]** Block Tags

Block Tag	Description
<code>_[[INDENT]]</code> spaces (4 default)	[3] Indent text
<code>_[[ITALIC]]</code> spaces (4 default)	[4] Indent italic
<code>_[[ENDNOTES]]</code> optional label	[5] Endnotes
<code>_[[TEXT]]</code> optional language	[6] Text
<code>_[[TOPIC]]</code> topic	[7] Topic
<code>_[[END]]</code>	[13] End block

[4i] Commands

Command	Description
IMAGE relative path scale, caption, figure	[5] Image file
IMAGE2 rel path1, rel path2 s1, s2, c1, c2, fig1, fig2	[6] Adjacent images

Command	Description
TEXT relative path language	[3] Text file
TABLE rel path title, width, rows, align, head	[4] Table file

D.7 Values - rv.V

[1i] rv.V Markup

The *Values* API function defines values and evaluates equations and functions.

[2i] Line Tags

Format a line of text

Line Tag	Description
text _ [V] var_name text	[11] Variable value
label _ [E]	[12] Equation number
title _ [I]	[13] Table number
caption _ [F]	[14] Figure number

[3i] Block Tags

Format or execute a block of text

Block Tag	Description
_ [[PYTHON]] namespace	[9] Python
_ [[END]]	[13] End block

[2i] Commands

Format files and calculations

Command	Description
TABLE rel path title,width,rows,align,head,num	[4] Table file
IMAGE relative path scale, caption, figure	[5] Image file

Command	Description
IMAGE2 rel path1, rel path2 c1, c2, s1, s2, f1, f2	[6] Adjacent images
VALTABLE rel path title, width, rows	[7] Valtable file
PYTHON relative path namespace	[11] Python file
a ==: 1*IN unit1, unit2, decimal, num label	[8] Define value
c <=: expression unit1, unit2, decimal, num label	[9] Assign value
c :=: expression unit1, unit2, decimal label, number	[9] Function value
a < c decimal text1,text2,align, num	[10] Compare values

D.8 Tool - rv.T

[1i] rv.T Markup

The *Tools* API function executes markup and scripts.

[2i] Block Tags

Block Tag	Description
_[[PYTHON]] namespace	[9] Python
_[[MARKUP]] type	[10] Markup block
_[[END]]	[13] End block

[3i] Commands

Command	Description
PYTHON relative path namespace	[11] Python file
MARKUP relative path type	[12] Markup file

D.9 Doc - rv.D

[1i] rv.D Markup

The *Doc API* publishes formatted *docs* from the rivt API strings.

[2i] Block Tags

Block Tag	Description
_ [[LAYOUT]] label	[12] Layout block
_ [[METADATA]] label	[11] Metadata

[3i] Commands

Command	Description
PDFATTACH relative path place, cover	[13] Attach PDF
PUBLISH ini rel. path type	[14] Publish doc

E.1 Collaboration

[1] Overview

There are several ways to collaborate on *rivt files* by using tools available in the Python, GitHub and VSCode ecosystem.

[2] Open Models

1. Public *rv*t files may be shared by uploading them to repositories e.g. *GitHub*. Public *rv*t files can be found and downloaded using the search interface. *Public rv*t files are identified with the naming convention

```
rv-DocNumber-filename.py
```

where the doc name and number are the doc number of the corresponding *doc* e.g. with an added hyphen prefix. The file is written to the *_shared* folder. This is in contrast with the form:

```
rvDocNumber-filename.py
```

for active, private files stored in the report root directory. After downloading shared *rv*t files, use a *batch rename* extension or utility to delete the hyphen and convert it to an active *rv*t file form. *Public rv*t files may be collaboratively improved by:

- forking to create different versions.
 - providing pull requests.
 - submitting issues and bugs.
2. Contributions to *rv*tlib at [rvtlib-dev](#) and documentation at [rvt-info](#) in the form of;
 - report issues and bugs
 - issue pull requests to propose changes
 - improved documentation
 3. Contributions to extensions and scripts that improve the interaction of *rv*t with other components in the *rv*t framework.
 4. Collaborative editing in *VSCode* using [Visual Studio Live Share](#).

[2] Open Models

The web site [OpenModels.info](#) is a *Google Drive* repository of open source engineering model files, including *rv*t files and reports.

[3] Git and GitHub

[Git](#) and [GitHub](#) are part of the *rv*t framework. *Git* is a free, open-source, distributed version control system designed to manage and track changes in files. It allows multiple people to work on the same project simultaneously without overwriting each other's work. *GitHub* is a web-based platform for hosting, managing, and collaborating on code built around *Git*. It allows teams to work together efficiently on software projects and provides features like pull requests, issue tracking, and project management.

Each *rv*t project is typically stored in its own repository. Every user of *GitHub* has a personal account with essentially unlimited repositories. Free accounts provide for:

- Unlimited Repositories: You can create as many public and private repositories as you need.
- Unlimited Collaborators: There is no limit to the number of people you can work with on your repositories.
- Community Support: You have access to the *GitHub* Community Discussions for help.
- Core Services with Usage Limits: The free plan includes a certain amount of monthly usage for services:

- **GitHub Actions: 2,000 minutes per month for private repositories**
(unlimited for public repositories).
- GitHub Codespaces: 120 core hours and 15 GB of storage per month.

GitHub Codespaces is a VSCode cloud-based development environment connected to GitHub files. It can be set up with *rivt extensions* that provide a complete rivt editing and collaboration environment in the cloud.

[4] VSCode

VSCode is an open source IDE with a large set of extensions, including collaboration support. Collaboration is facilitated by the [Visual Studio Live Share](#). You can start a *Live Share* session either within *CodeSpaces* in your browser, or within the *VS Code* desktop application. *CodeSpaces* is a cloud based GitHub implementation of VSCode that shares many of the same features.

The *Live Share* extension enables:

- Real-time Co-editing: Multiple participants can work on the same file simultaneously, seeing each other's cursors, selections, and edits instantly, much like in Google Docs..
- Shared Terminals and Servers: The host can share their terminal and localhost servers with guests, eliminating the need for guests to set up their own environments or dependencies.
- Navigation and Following: Participants can independently navigate the project or choose to "follow" another collaborator, keeping their editor synchronized with the leader's actions.
- File Access Control: Hosts can use a `.vsls.json` file to control which files and folders are visible or editable by guests, ensuring security and focus

E.2 Public files

[1] Overview

A feature of *rivt* is fine-grained control over public sharing of all or parts of a rivt file. *Public rivt files* are generated by *rivtlib* on a section by section basis defined by the API header substring. Each API function defines a section and has a public/private setting in the header. Sections are private by default (see `:doc: <rvF01-quickref>`).

```
rv._("""Section Label | doc;stored, private;public, section;merge
    Content substring indented 4 spaces
    ...
    """)
```

Sections are written to a rivt file in the `/public` folder with a file name composed as:

```
rivt file name
    rvAnn-filename.py
modified with an additional hyphen
```

```
rv-Ann-filename.py
```

The public folder may be uploaded to a GitHub repo for public sharing. When downloaded the extra hyphens may be removed with a batch remove command (e.g. in VSCode).

[2] Public Folder

For each rvt file that is published a corresponding *public rvt file* is written to the *public* folder. The default contents of the file are the API rvt string headers. If the header parameter *public* is included in the header, the rvt string content is also written to the *public rvt file*.

[rvt]-Report-Label/	Report Folder Name
[rv101-]filename1.py	rvt file
[rv102-]filename2.py	rvt file
[rv201-]filename3.py	rvt file
[rv202-]filename4.py	rvt file
...	
[public]/	public rvt files
rv-101-filename1.py	
rv-102-filename1.py	
rv-201-filename3.py	
rv-202-filename4.py	

The *public folder* can be uploaded to a public repository and updated with Git as the project develops.

[3] Example

E.3 Git

[1] Git

GitHub is a popular and widely used cloud and version control system based on the Git program that may be downloaded [here](#).

E.4 GitHub Search

[1] GitHub Global Search

This page is a search interface for **rvt** files on GitHub. The search covers the full text of the **rvt** README.txt file in the *repo* directory. The README file contains the *rvt report*. For advanced searching use the [GitHub interface](#).

[S] Search GitHub <Ctrl+Enter>

[C] Clear input <Ctrl+R>

Example: solar+steel+frame is passed to GitHub as rivt+solar+steel+frame

[2] GitHub Organization Search

Restrict search to the following comma separated GitHub organizations:

F.1 Quick Ref

[1] API

API functions

Function	Name	Purpose
rv.R (rS)	Run	Run scripts and markup
rv.I (rS)	Insert	Insert static sources
rv.V (rS)	Values	Calculate values
rv.T (rS)	Tools	Execute shell commands and programs
rv.D (rS)	Doc	Publish docs
rv.S (rS)	Skip	Skip section (comments and debugging)
rv.X (rS)	Exit	Exit rivt (debugging)

where **rS** is a *rivt string*. The first line of a *rivt string* (rS) is the header substring.

Header substrings

```
rv.I("""A New Section | private, doc, section
    Content text
    ...
    """)
```

Default settings in the *header substring* do not need to be specified. The default setting for each API is listed first (in bold) in the table below. The default privacy settings for all sections in a rivt file may be reversed by including the *public setting* and setting it to True, immediately following the *rivlib import statement*. Individual sections may still be set as private in the header substring.

```
import rivtlib.rvapi as rv
# rv public=True
```

Header substring defaults

[2] Line Tags

API	private; public	doc; stored	section; merge
rv.R	private ; public	stored ; doc	merge ; section
rv.I	private ; public	doc ; stored	section ; merge
rv.V	private ; public	doc ; stored	section ; merge
rv.T	private ; public	stored ; doc	merge ; section
rv.S	private ; public	stored	merge
rv.D	public	stored	merge
rv.X	•	•	•

[2] Line Tags

Format a line of text

API Scope	Line Tag	Description
rv.I,V	title _[]	[13] Table number
rv.I,V	caption _[F]	[14] Figure number
rv.I,V	text _[C]	Bold center text
rv.I,V	text _[R]	Bold right text
rv.I,V	text math _[M] description	[4] Text math
rv.I,V	LaTeX math _[L] description	[5] LaTeX math
rv.I,V	text _[#] text	[6] Endnote number
rv.I,V	text _[G] glossary term] text	[7] Glossary link
rv.I,V	text _[S] label, section] text	[8] Section link
rv.I,V	text _[U] label, url] text	[10] URL link
rv.I,V	text _[V] var_name] text	[11] Variable value
all	## text	nonprinting comment

[3] Block Tags

Format blocks of text or run code

[4] Commands**Commands for reading and writing files**

API Scope	Command	Description
rv.R	MARKUP rel path type	[12] Markup file
rv.V, l	TABLE rel path title,width,rows,	[4] Table file
rv.V, l	IMAGE rel path caption, scale, num;non	[5] Image file
rv.V, l	IMAGE2 rel pth1, rel pth2 cap1,cap2,sc1,sc2,num,num	[6] Adjacent images
rv.V	VALTABLE rel path title, rows	[7] Valtable file
rv.V	PYTHON rel path description	[11] Python file
rv.V	a ==: 1*IN unit1, unit2, decimal description	[8] Define value
rv.V	c <=: expression unit1, unit2, decimal desc	[9] Assign value
rv.V	c :=: function(x,y) unit1, unit2, decimal desc	[9] Function value
rv.V	a < c unit, decimal, text1, text2 description	[10] Compare values
rv.T	SHELL rel path os, wait;nowait	[2] Shell file
rv.D	ATTACHPDF rel path place, title	[13] Attach PDF
rv.D	PUBLISH doc title type	[14] Publish doc

Relative file paths for commands

See here for the full folder structure. File paths are relative to the *rivt-report* folder. Use *_stored/filename* to read values or files defined and written by the *rivt* file.

Command	Standard Path
SHELL	src/filename

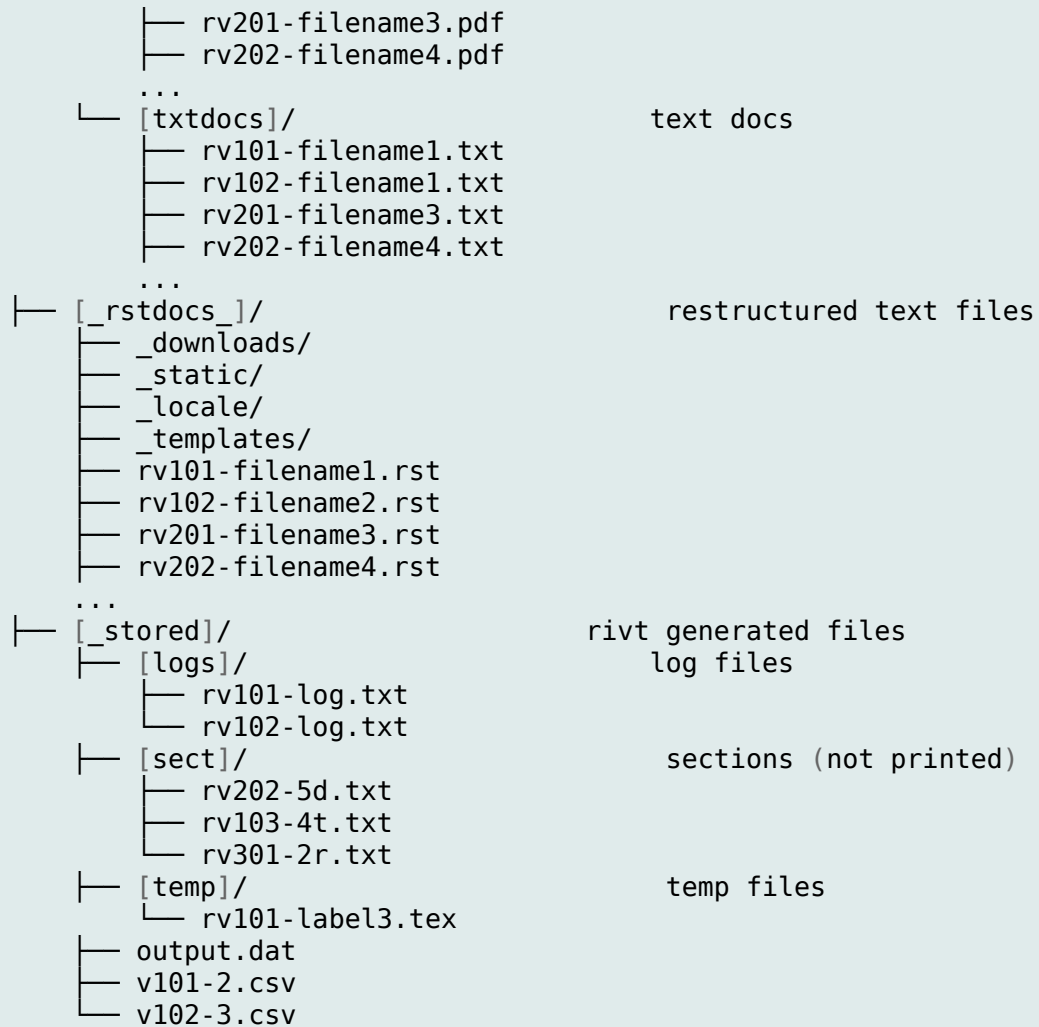
Command	Standard Path
TABLE	src/filename
IMAGE	src/filename
IMAGE2	src/filename
VALTABLE	src/filename
PYTHON	src/filename
MARKUP	src/filename
ATTACHPDF	src/filename
PUBLISH	docs written to _publish

[5] Folders

```

[rixt-]Report-Label/          Report Folder
├── .vscode/                  optional VSCode settings
├── README.txt                rixt-generated report
├── [_rixt-public]/          rixt-generated public files
│   ├── _src/                source files
│   ├── README.txt           public report
│   ├── rv-101-filename1.py   public rixt file
│   ├── rv-102-filename2.py   public rixt file
│   ├── rv-201-filename3.py   public rixt file
│   └── rv-202-filename4.py   public rixt file
├── ...
└── [rixt-report]/           rixt files and docs
    ├── [rv101-]filename1.py   rixt file
    ├── [rv102-]filename2.py   rixt file
    ├── [rv201-]filename3.py   rixt file
    └── [rv202-]filename4.py   rixt file
    ├── ...
    ├── [src]                 author source files
    │   ├── data/             user folder (tables, etc.)
    │   │   ├── opensees1.txt
    │   │   └── conc-vals.csv
    │   ├── image/
    │   │   ├── fig1.png
    │   │   └── fig2.jpg
    │   ├── tool/             OS shellcommands
    │   │   ├── run1_win.cmd
    │   │   └── run1_linux.sh
    │   ├── run/              markup and functions
    │   │   ├── coverpage.rst
    │   │   ├── logoname.png
    │   │   ├── plot.py
    │   │   └── loads.py
    │   ├── steel-vals.csv
    │   └── plastic-vals.csv
    ├── [_published]/         published docs and reports
    │   ├── [docs]/           html docs
    │   │   ├── _images/
    │   │   ├── _sources/
    │   │   ├── _static/
    │   │   ├── site folders/
    │   │   ├── process folders/
    │   │   ├── rv101-filename1.html
    │   │   ├── rv102-filename2.html
    │   │   ├── rv201-filename3.html
    │   │   └── rv202-filename4.html
    │   ├── ...
    │   └── [pdfdocs]/        pdf docs
    │       ├── process folders/
    │       ├── rv101-filename1.pdf
    │       └── rv102-filename1.pdf

```



[6] Python Requirements

The minimum Python version is 3.14. *rivt* project packages include the following categories:

- formatting
- numerical analysis
- symbolic processing
- visualization
- IDE

Dependency	description
"pyzo>=4.20.0"	lightweight IDE
"pyside6>=6.10.1"	QT bindings
"fastcore>=1.8.16"	code simplification
"tabulate>=0.9.0"	format tables

Dependency	description
"pillow>=11.2.1"	image processing
"matplotlib>=3.10.1"	data visualization
"sympy>=1.13.3"	symbolic analysis
"numpy>=2.2.5"	numerical analysis
"scipy>=1.16.3"	numerical analysis
"pandas>=2.2.3"	data analysis
"docutils>=0.21.2"	reStructuredText processing
"ipython>=8.16.2"	interactive Python shell
"ipykernel>=6.28.1"	Jupyter kernel for Python
"reportlab>=4.4.0"	PDF generation without LaTeX
"rst2pdf>=0.103.1"	PDF generation without LaTeX
"pypdf>=1.0.3"	PDF manipulation
"Sphinx>=8.2.3"	HTML generation
"pydata-sphinx-theme>=0.16.1"	HTML generation
"sphinx-copybutton>=0.5.2"	HTML generation
"sphinx_design>=0.6.1"	HTML generation
"sphinx-favicon>=1.0.1"	HTML generation
"sphinx-togglebutton>=0.3.2"	HTML generation
"sphinxcontrib-applehelp>=2.0.0"	HTML generation
"sphinxcontrib-devhelp>=2.0.0"	HTML generation
"sphinxcontrib-email>=0.3.6"	HTML generation
"sphinxcontrib-htmlhelp>=2.1.0"	HTML generation
"sphinxcontrib-jquery>=4.1"	HTML generation
"sphinxcontrib-jsmath>=1.0.1"	HTML generation
"sphinxcontrib-qthelp>=2.0.0"	HTML generation
"sphinxcontrib-serializinghtml>=2.0.0"	HTML generation

[7] VSCode Settings

Workspace extension and other settings are stored in the `.vscode` folder and should be included as part of a *rivt* folder. Settings that affect the *rivt* editing and execution environment include:

- `rivt.code-snippets`
- `settings.json` (extension settings)
- `tasks.json`
- `launch.json`

Global settings can be exported and imported using VSCode Profiles. VSCode *rivt* extensions and settings may be installed from this link.

Extension	description
BUTTONS	—
tombonnike.vscode-status-bar-format-toggle	format button
gsppvo.vscode-commandbar	command buttons
AdamAnand.adamstool	command buttons
nanlei.save-all	save all button
Ho-Wan.setting-toggle	toggle settings
yasukotelin.toggle-panel	toggle panel
fabiospampinato.vscode-commands	user command buttons
jerrygoyal.shortcut-menu-bar	menu bar
EDITING	—
henryclayton.context-menu-toggle-comments	toggle comments
TroelsDamgaard.reflow-paragraph	wrap paragraph
streetsidesoftware.code-spell-checker	spell check
jmviz.quote-list	quote elements in a list
njpwerner.autodocstring	insert doc string
oijaz.unicode-latex	unicode symbols from latex
jsynowiec.vscode-insertdatestring	insert date string
janisdd.vscode-edit-csv	csv editor
VIEWS	—
GrapeCity.gc-excelviewer	excel viewer
SimonSiefke.svg-preview	svg viewer

Extension	description
tomoki1207.pdf	pdf viewer
RandomFractalsInc.vscode-data-preview	data viewing tools
Fr43nk.seito-openfile	open file from path
vikyd.vscode-fold-level	line folding tool
file-icons.file-icons	icon library
tintinweb.vscode-inline-bookmarks	inline bookmarks
MANAGEMENT	—
alefragnani.project-manager	folder, project management
Anjali.clipboard-history	clipboard history
dionmunk.vscode-notes	notepad
hbenl.vscode-test-explorer	test explorer
mightyco.co.fsdeploy	save file to second location
lyzerk.linecounter	count lines in files
sandcastle.vscode-open	open files in default app
zjffun.snippetsmanager	snippet manager
spmeesseman.vscode-taskexplorer	task explorer
GITHUB	—
GitHub.codespaces	run files in codespaces
GitHub.remotehub	run remote files
ettoreciprian.vscode-websearch	search github within VSCode
donjayamane.git-history	git history
MichaelCurrin.git-commit-msg	git auto commit message
github.vscode-github-actions	github actions
GitHub.vscode-pull-request-github	github pull request
k9982874.github-gist-explorer	gist explorer
vs-libs-contrib.gistfs	gist tools
PYTHON	—
ms-python.autopep8	python pep8 formatting

Extension	description
ms-python.isort	python sort imports
donjayamanne.python-environment-manager	python library list
ms-python.python	python tools
ms-python.vscode-pylance	python language server
ms-toolsai.jupyter	jupyter tools
ms-toolsai.jupyter-keymap	jupyter tools
ms-toolsai.jupyter-renderers	jupyter tools
ms-toolsai.vscode-jupyter-cell-tags	jupyter tools
ms-toolsai.vscode-jupyter-slideshow	jupyter tools
LANGUAGES	—
qwtel.sqlite-viewer	sqlite tools
RDebugger.r-debugger	R tools
REditorSupport.r	R tools
ms-vscode-remote.remote-wsl	windows linux tools
James-Yu.latex-workshop	latex tools
lextudio.restructuredtext	restructured text tools
trond-snekvik.simple-rst	restructured syntax
yzane.markdown-pdf	markdown to pdf
yzhang.markdown-all-in-one	markdown tools

Snippets and extensions

Keystrokes	Description
rvr	API Run
rvi	API Insert
rvv	API Values
rvt	API Tools
rvd	API Docs
alt+ctrl+c	insert cell marker with label above API function

Keystrokes	Description
alt+q	rewrap paragraph with hard line feeds (80 default)
alt+p	open file under cursor
alt+.	select correct spelling under cursor
alt+8	insert date
alt+9	insert time
ctl+1	focus on first editor
ctl+2	focus on next editor
ctl+3	focus on previous editor
ctl+8	focus on explorer pane
ctl+9	focus on github pane
ctl+alt+x	reload window
ctl+alt+[	reload window
ctl+alt+]	unfold all code
ctl+alt+u	unfold all code
ctl+alt+f	fold code level 2 (rivt sections visible)
ctl+alt+a	fold code - all levels
ctl+alt+t	toggle local fold
ctl+alt+e	toggle explorer sort order
ctl+alt+s	toggle spell check
ctl+alt+g	next editor group
ctl+shift+u	open URL under cursor in browser
ctl+shift+s	open GitHub README search for rivt
ctl+shift+a	commit all
ctl+shift+z	commit the current editor
ctl+shift+x	post to remote

Modified Keyboard Shortcuts

Keystroke	Description
alt+q	rewrap paragraph with hard line feeds (80 default)
alt+p	open file under cursor
alt+.	select correct spelling under cursor
alt+8	insert date
alt+9	insert time
ctl+1	focus on first editor
ctl+2	focus on next editor
ctl+3	focus on previous editor
ctl+8	focus on explorer pane
ctl+9	focus on github pane
ctl+alt+x	reload window
ctl+alt+u	unfold all code
ctl+alt+f	fold code level 2 (rivt sections visible)
ctl+alt+a	fold code - all levels
ctl+alt+t	toggle local fold
ctl+alt+e	toggle explorer sort order
ctl+alt+s	toggle spell check
ctl+alt+g	next editor group
ctl+shift+u	open URL under cursor in browser
ctl+shift+s	open GitHub rivt README search
ctl+shift+a	commit all
ctl+shift+z	commit current editor
ctl+shift+x	post to remote

F.2 Terms

[1] rivt Terms

block tag : block tags

A block tag formats a block of text that begins with `_[[TAG]]_` and terminates with `_[[Q]]_`.

command : commands

rivt commands read and write external files and assign values to variables. They start in the initial columns with a vertical bar (|) followed by the file path, name and parameters. The path is relative to the rivt report root folder. If the metadata variable *rv_localB* is set to true resources will be read from and written to the local *rivt file* folder.

```
| COMMAND | relative path | parameters
```

content : content text : content substring

The text in a *rivt string* that follows the *header substring*. The text is utf-8 (Unicode Transformation Format - 8-bit), a character encoding standard that is a superset of ASCII, handles text from any language, and is constant across all platforms.

division

A group of related rivt files organized in a report and labeled in the rivt file name. Optionally organized in a division folder.

doc : docs

A formatted document file output using the rivtlib package. A doc file is a text, PDF or HTML file with the name taken from the corresponding rivt file.

docnumber : doc number : doc numbers

Prefix number of a rivt and doc file name used to organize a report. It has the form *rvDss-filename.py* where D is a capital alphanumeric division number and nn is the subdivision number.

header : headers : header text : header substring

The first line of a *rivt string* that includes a section title, followed by comma separated parameters.

line tag : line tags

A line tag has the form *_[TAG]* and formats a line of text.

report : reports : rivt report : rivt reports

A group of compiled *docs* organized by rivt file number.

report folder : report folders

The folder structure for producing a report is described here.

report script

A Python script that assembles *docs* into a *report*.

rivt

An open source Python project that includes rivtlib and approximately two dozen Python packages. When *rivt* is installed docs and reports may be edited and published using a text editor.

rivt doc : rivt docs

A formatted document file output from a *rivt file* using the rivtlib package. A doc is a text, PDF or HTML file with the default name of the *rivt file* and file type suffix.

rivt file : rivt files

A Python file that imports *rivtlib* and includes *rivt markup*.

rivt file number

Prefix of rivt file name used to organize a report (rvDss-). It has the form *rvDss-filename.py* where D is a capital alphanumeric division number and ss is the subdivision number.

rivt folder

A folder containing a rivt file and all of its resources in the same folder. Generally used

rivt framework : framework

The recommended *rivt-framework* includes the following separately installed programs:

1. *VSCode* <<https://code.visualstudio.com/>> and *VSCode extensions* <<https://marketplace.visualstudio.com/vscode>>. *rivt* extensions and snippets are here.
2. *Git* <<https://git-scm.com>> and set up a *GitHub* <<https://github.com>>
3. *LaTeX* <<https://www.tug.org/texlive/>>
4. *QCAD* <<https://qcad.io/en/>>

rivt log file

rivt file execution log written to the *log* folder as *rvDss-log.txt*.

rivt markup

A light weight markup that wraps restructuredText and outputs reports in text, PDF and HTML. Markup is included in string arguments to rivt API functions.

rivt report folder

A folder containing multiple rivt files to be organized in a report and resources organized in subfolders

rivt string

Triple quoted string argument to an API function.

rivtlib

Open source [Python package](#) that generates docs and reports from a rivt file

rvspace

rivt alias name for the global rivt file namespace. Other user namespaces may be specified

section parameter : section parameters

Comma separated parameters in a *header* that specify the section processing.

[2] Python Terms

docutils

A Python package that processes [restructured text](#) files into HTML, LaTeX, and other formats.

namespace

Provides [scope](#) for functions and variables.

Python : python

A [programming language](#) that lets you work quickly and integrate systems more effectively

restructuredtext : restructuredtext markup

A lightweight markup language designed to be processed by document software including [docutils](#), Sphinx and rivt.

[3] GitHub and VSCode Terms

fork : forked

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project. Most commonly, forks are used to either propose changes to someone else's project or to use someone else's project as a starting point for your own idea.

Git : git

A [version control system](#) that lets you manage and keep track of your source code history.

IDE : ide

Integrated Development Environment. A software application that provides comprehensive facilities to computer programmers for software development. An IDE typically includes a code editor, build automation tools, and a debugger. *rivt extensions* have been developed for VSCode. Other IDEs include PyCharm, Spyder, and JupyterLab.

profile

Allows VSCode users to customize their environment for different workflows, projects, or tasks. This feature provides a way to manage distinct configurations of settings, extensions, keyboard shortcuts, snippets, and tasks.

repo : repository

a storage location for software packages

rivt Codespace : rivt codespace

A Codespace set up with extensions, snippets and keyboard shortcuts for editing and publishing *rivt docs*.

Index

B

- [block tag](#)
- [block tags](#)

C

- [command](#)
- [commands](#)
- [content](#)
- [content substring](#)
- [content text](#)

D

- [division](#)
- [doc](#)
- [doc number](#)
- [doc numbers](#)
- [docnumber](#)
- [docs](#)
- [docutils](#)

F

- [fork](#)
- [forked](#)
- [framework](#)

G

- [Git](#)
- [git](#)

H

- [header](#)
- [header substring](#)
- [header text](#)
- [headers](#)

I

- [IDE](#)
- [ide](#)

L

- [line tag](#)
- [line tags](#)

N

- [namespace](#)

P

- [profile](#)
- [Python](#)
- [python](#)

R

- [repo](#)
- [report](#)
- [report folder](#)
- [report folders](#)
- [report script](#)
- [reports](#)
- [repository](#)
- [restructuredtext](#)
- [restructuredtext markup](#)
- [rivt](#)
- [rivt Codespace](#)
- [rivt codespace](#)
- [rivt doc](#)
- [rivt docs](#)
- [rivt file](#)
- [rivt file number](#)
- [rivt files](#)
- [rivt folder](#)
- [rivt framework](#)
- [rivt log file](#)
- [rivt markup](#)

- [rivt report](#)
- [rivt report folder](#)
- [rivt reports](#)
- [rivt string](#)
- [rivtlib](#)
- [rvspace](#)

S

- [section parameter](#)
- [section parameters](#)